

高等計算機演算法

term project

學號：M11202169

姓名：鍾坤佑

Design sorting network (bitonic sorter)	3
1. Input Module (SIPO_16x8)	3
2. Output Module (PISO_16x8)	4
3. 8 bits Compare Swap Unit	5
4. Bitonic Merger 4 inputs	5
5. Bitonic Merger 8 inputs	6
6. Bitonic Merger 16 inputs	6
7. Bitonic Sorter 16 inputs	7
8. Datapath	8
9. Controller	8
10. Complete Sorter 16 inputs 8bits each	9
Verilog Code and Simulation	9
1. Input Module (SIPO_16x8)	9
2. Output Module (PISO_16x8)	13
3. 8 bits Compare Swap Unit	15
4. Bitonic Merger 4 inputs	18
5. Bitonic Merger 8 inputs	22
6. Bitonic Merger 16 inputs	25
7. Bitonic Sorter 16 inputs	30
8. Datapath	33
9. Controller	36
10. Complete Sorter 16 inputs 8bits each	39
Performance	42
Power Consumption	42

Hardware Cost	43
Discussion	44

Design sorting network (bitonic sorter)

這次的 project 我選擇實作 A Sorting Network with 16 inputs (8-bit each)。在整體架構設計中，首先透過 SIPO_16x8 (Serial-In Parallel-Out) 模組，將輸入資料分批傳入，每次輸入 8 bits，連續傳送 16 次後即可完成所有輸入資料的收集。接著，核心的排序運算由 Bitonic Sorter 模組負責，該模組內部由多層 Compare-and-Swap Unit (CS) 組成，依據 Bitonic 排序原理進行階層式比較與交換操作。最後，排序完成的結果會輸入至 PISO_16x8 (Parallel-In Serial-Out) 模組，將資料分批輸出，每次輸出 8 bits，連續輸出 16 次即可完成，順序由最小值 (min) 至最大值 (max)。

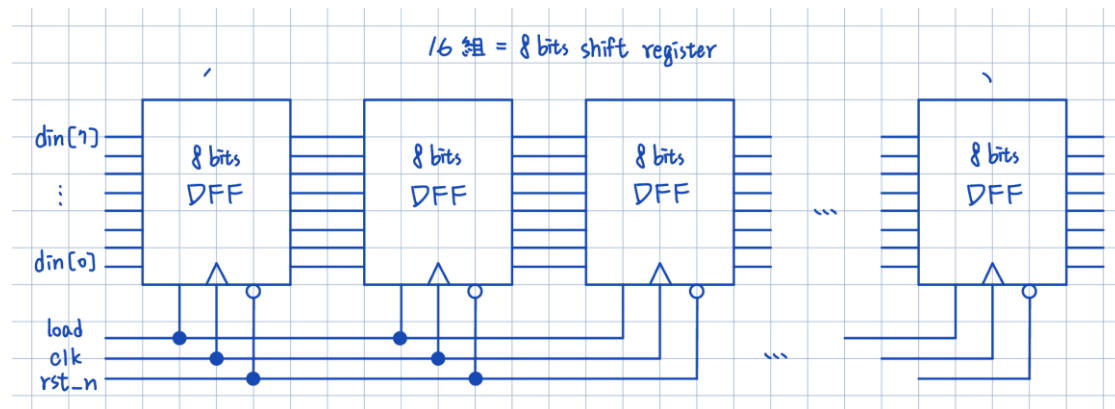
在內部運作上，Bitonic Sorting Network 屬於一種固定結構的比較網路。由於整個連線在設計階段即已確定，因此輸入資料的排列方式與運算週期皆為固定。此外，該架構能在每個 clock cycle 內同時執行多組 Compare-and-Swap 操作，因此具備高度的平行性，適合以硬體實現高效排序。

在本設計中，16-input 的 Bitonic Sorting Network 完成排序需經過 10 個 clock cycle。將前後的資料輸入與輸出考慮後，整體運作流程 (SIPO 輸入、Sorter 排序、PISO 輸出) 共需 $16 + 10 + 16 = 42$ 個 clock cycle 完成一次完整的排序作業。

1. Input Module (SIPO_16x8)

首先，由於輸入輸出的 pin 腳數量限制在 32 個以內，因此在設計上，必須用分批輸入的方式，讓系統能在有限的 I/O 數量下完成 16 組資料的接收。在我的設計中，SIPO_16x8 模組負責將外部資料依序輸入，每次傳入 8 bits，連續傳送 16 次後就可以完成所有輸入資料的收集。

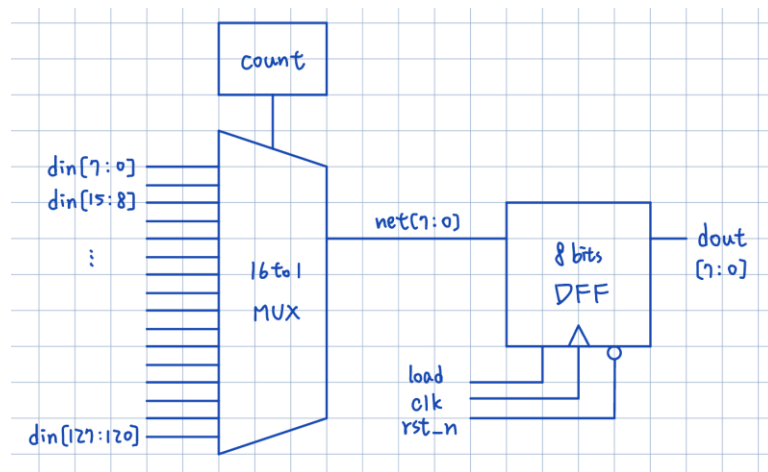
在硬體架構上，我採用 Shift Register 的概念。每當新的一筆資料輸入時，暫存器內的內容會整個移位，新的 8bits 資料則會被填入最低位的位置。連續移位 16 次後，128 bits (16 x 8 bits) 的資料就準備完成，每 8 bits 即對應一筆要排序的輸入資料。



2. Output Module (PISO_16x8)

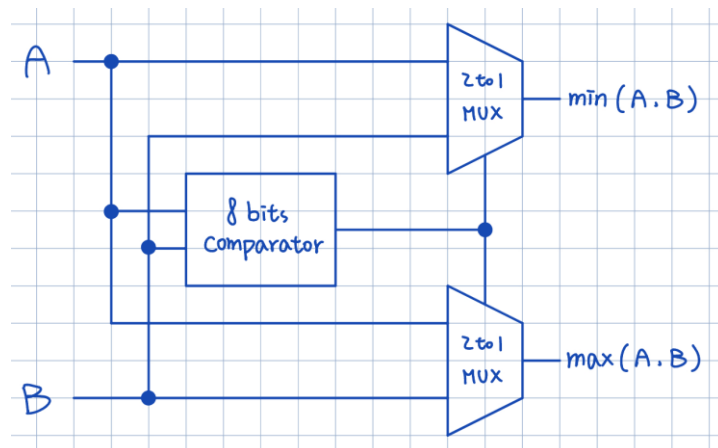
在輸出端的設計中，也因為輸入輸出 pin 腳數量限制的關係，PISO_16x8 模組負責將排序完成後的 128 bits 資料依序輸出，這裡我也是用分批輸出的方式，每次輸出 8 bits，連續輸出 16 次後即可完成。

在硬體架構上，由於排序完成後，已經有一組 128 bits 的暫存器儲存所有排序完的資料，所以現在就只需要由低至高依序將 8bits 的資料傳出去就可以了，這裡我利用計數器配合 16 對 1 的 8bits 多工器，經過 16 個 clock cycle 依序將資料傳出去。



3. 8 bits Compare Swap Unit

處理完輸入輸出模組後，接著要來組一個”比較交換”的基本單元 Compare Swap Unit，這個基本單元是由一個比較器加上兩個多工器所組成的，由於我們每一個比較的資料是 8 bits，所以需要一個 8 bits 的比較器加上兩個 8 bits 的二對一多工器，有了這個基本單元後，之後的模組就可以由這個基本單元組出來。

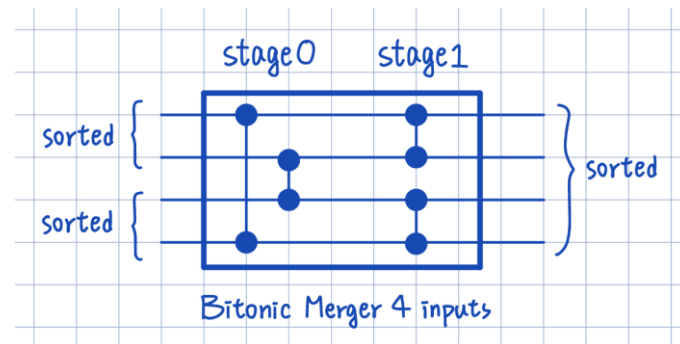


4. Bitonic Merger 4 inputs

由於我們已經有了一個 8 bits 的 Compare Swap Unit，而這個基本單元可以處理兩個 inputs 的比較交換操作，所以也等同是 Bitonic Merger 2 inputs 的模組，現在要開始慢慢擴大，把 input 數量最終擴大到 16 inputs，這裡的每一個子模組在最後都會合併成一個 Bitonic Sorter 16 inputs。

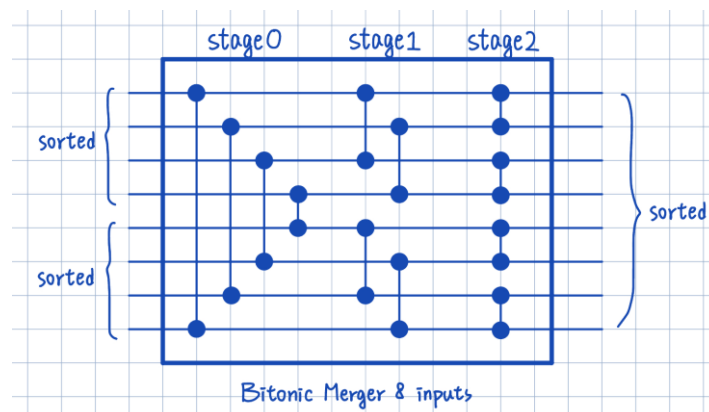
這裡我先用 8 bits 的 Compare Swap Unit 接出一個 Bitonic Merger 4 inputs，如下圖可以看到這個模組因為有四個 8 bits 的 Compare Swap Unit，但是只需要

花費兩個 stage (2 clock cycle)，所以可以得知每個 stage 內都有兩個 8 bits 的 Compare Swap Unit 在運作，也就是硬體發揮他的的平行處理能力。



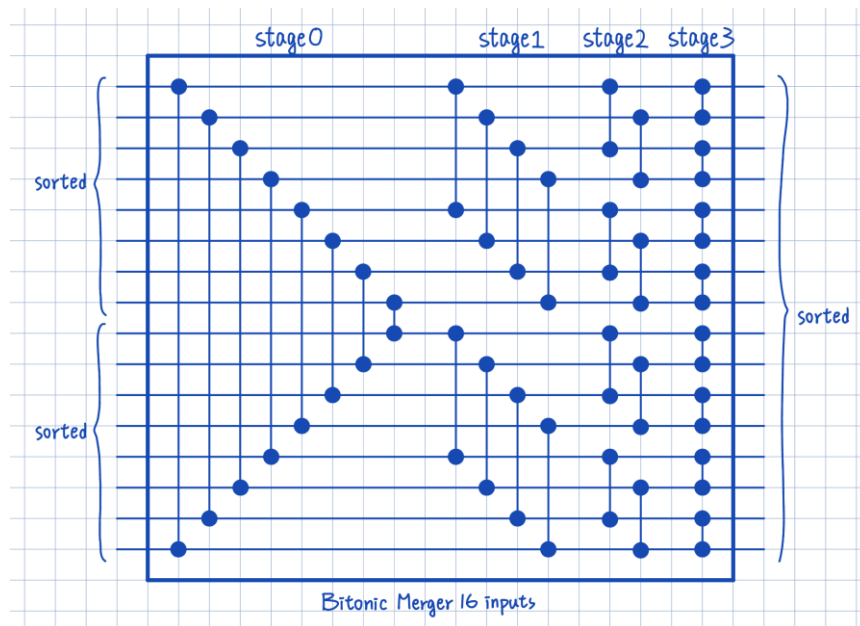
5. Bitonic Merger 8 inputs

接著因為最終需要處理 16 inputs 的資料排序，所以我們還是要繼續擴大，下圖是我接的 Bitonic Merger 8 inputs，可以看到原本 4 inputs 只需要兩個 stage，現在 8 inputs 需要花三個 stage，且每個 stage 內同時有四個 8 bits 的 Compare Swap Unit 在運作。



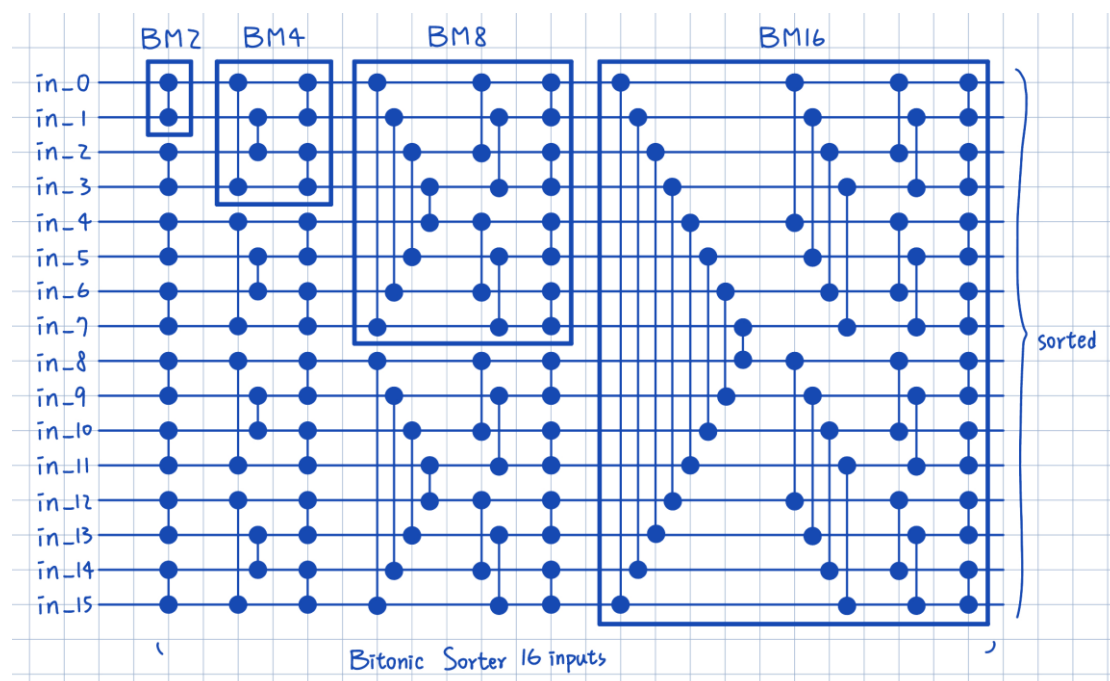
6. Bitonic Merger 16 inputs

現在為了要完成最終的 Bitonic Sorter 16 inputs，我們要來接最後的子模組 Bitonic Merger 16 inputs，下圖是我接的 Bitonic Merger 16 inputs，可以看到現在 16 inputs 的版本需要花四個 stage，且每個 stage 內同時有八個 8 bits 的 Compare Swap Unit 在運作。



7. Bitonic Sorter 16 inputs

現在所有子模組都有了，就可以把之前的子模組結合成一個 Bitonic Sorter 16 inputs，根據課本講義所學，可以知道一個 N input 的 Bitonic Sorter 是由 $1 \times (N \text{ input Bitonic Merger}) + 2 \times (N/2 \text{ input Bitonic Merger}) + 4 \times (N/4 \text{ input Bitonic Merger}) + \dots$ 加到 2 input Bitonic Merger，所以這些結合在一起就可以得到下圖的接法：

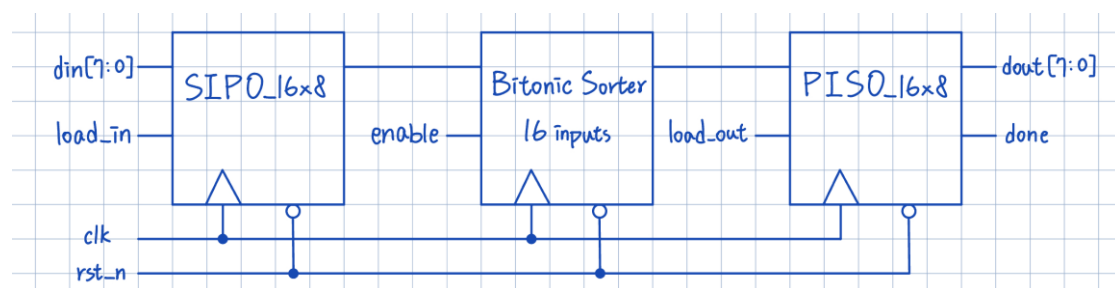


此時，這個 16 inputs 的 Bitonic Sorter 就完成了，總共由八個 Bitonic Merger

2 inputs + 四個 Bitonic Merger 4 inputs + 兩個 Bitonic Merger 8 inputs + 一個 Bitonic Merger 16 inputs，整個架構完成排序要花費 $1 + 2 + 3 + 4 = 10$ 個 stage 的時間也就是 10 個 clock cycle，每個 stage 內都會有八個 8 bits 的 Compare Swap Unit 同時在運作。

8. Datapath

完成前面的部分之後就可以把三個主要的模組 Input Module (SIPO_16x8)、Bitonic Sorter 16 inputs、Output Module (PISO_16x8) 給結合起來成為一個完整的 datapath，但此時的 datapath 的控制訊號還是由外部輸入的，因為我們還沒有完成 controller 的部分，所以在 testbench 上就必須花點時間處理控制訊號的部分，完整 datapath 如下圖：



9. Controller

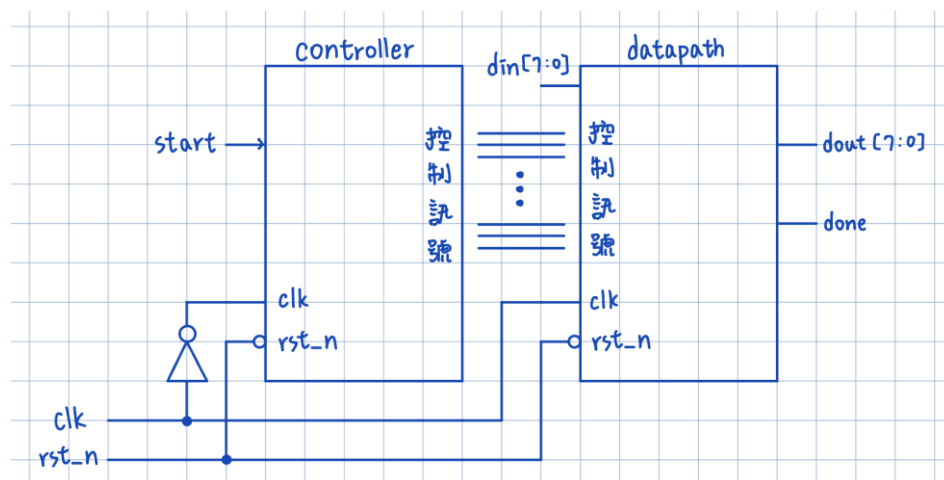
完成 datapath 之後，就可以來設計 controller 來控制原本 datapath 裡面的控制訊號，這次的 project 相較於 FPGA 所設計的 multi-cycle CPU 來說，控制訊號線少很多，而且並不會像 multi-cycle CPU 一樣，每個指令有不同的完成週期，這次的 Bitonic Sorter 16 inputs 有固定的運算週期為 10 個 clock cycle，另外，輸入輸出模組各需要 16 個 clock cycle 來完成，所以總共需要 42 個 clock cycle。

在 controller 的設計上，前 16 個 clock cycle 為 SIPO_16x8 的控制訊號，接著 10 個 clock cycle 為 Bitonic Sorter 16 inputs 的控制訊號，而後 16 個 clock cycle 為 PISO_16x8 的控制訊號，因為都是固定的週期與順序，所以並不需要像 multi-cycle CPU 一樣用 FSM 來寫，只需要簡單的 counter 用來輸出對應的控制訊號線即可。

收到 start 訊號後	⇒	0~15 clock cycle	$\left\{ \begin{array}{l} \text{load_in} = 1 \\ \text{enable} = 0 \\ \text{load_out} = 0 \end{array} \right.$
		16~25 clock cycle	$\left\{ \begin{array}{l} \text{load_in} = 0 \\ \text{enable} = 1 \\ \text{load_out} = 0 \end{array} \right.$
		26~41 clock cycle	$\left\{ \begin{array}{l} \text{load_in} = 0 \\ \text{enable} = 0 \\ \text{load_out} = 1 \end{array} \right.$

10. Complete Sorter 16 inputs 8bits each

設計完 datapath 與 controller 後就可以把這兩個模組合起來，變成最後完整的系統，總 I/O pin 腳控制在 20 個，有符合這次 project 的需求 (32 個以內)。



Verilog Code and Simulation

1. Input Module (SIPO_16x8)

首先，這個 SIPO_16x8 模組作為輸入端的設計，功能與 design 的描述一致，是將外部連續輸入的 8 bits 資料依序儲存，每輸入一次資料，暫存器內容便整體向高位移動，新資料填入最低位，經過 16 次輸入後，即可形成完整的 128 bits 資料。在 verilog 的寫法中，我簡單的以 load 控制訊號與計數器 count 來進行 16 次的資料傳遞，程式寫法如下所示。

Verilog :

```
1  `timescale 1ns / 1ps
2  //////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 2025/11/07 14:01:34
7  // Design Name:
8  // Module Name: SIPO_16x8
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 //////////////////////////////////////
21
22
23 module SIPO_16x8(
24     input wire clk,
25     input wire rst_n,
26     input wire load,
27     input wire [7:0] din,
28     output reg [127:0] dout,
29     output reg done
30 );
31
32
33     reg [3:0] count;
34     always @(posedge clk or negedge rst_n)begin
35         if(!rst_n)begin
36             dout <= 128'b0;
37             count <= 4'b0;
38             done <= 1'b0;
39         end
40         else begin
41             done <= 1'b0;
42             if (load)begin
43                 dout <= {dout[119:0], din};
44                 count <= count + 1'b1;
45                 if(count==4'd15)begin
46                     done <= 1'b1;
47                 end
48             end
49         end
50     end
51 endmodule
```

Testbench :

```

1  `timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 2025/11/07 14:44:00
7  // Design Name:
8  // Module Name: tb_SIPO_16x8
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21
22
23 module tb_SIPO_16x8;
24     reg clk;
25     reg rst_n;
26     reg load;
27     reg [7:0] din;
28     wire [127:0] dout;
29     wire done;
30
31     SIPO_16x8 uut (
32         .clk(clk),
33         .rst_n(rst_n),
34         .load(load),
35         .din(din),
36         .dout(dout),
37         .done(done)
38     );
39
40     always #10 clk = ~clk;
41     initial begin
42         clk = 0;
43         rst_n = 0;
44         load = 0;
45         din = 8'h00;
46
47         #100;
48         rst_n = 1;
49
50         // 依序送 16 個 byte
51         repeat (16) begin
52             @(negedge clk);
53             load = 1;
54             din = $random; // 隨機送一個 byte
55         end
56
57         @(negedge clk);
58         load = 0;
59
60         #100;
61         $finish;
62     end
63 endmodule

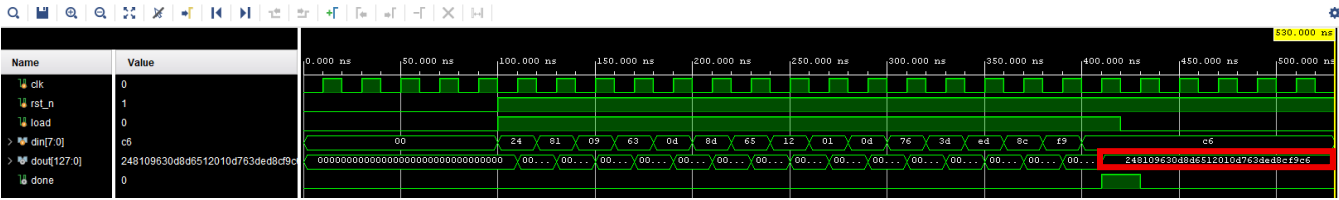
```

Simulation :

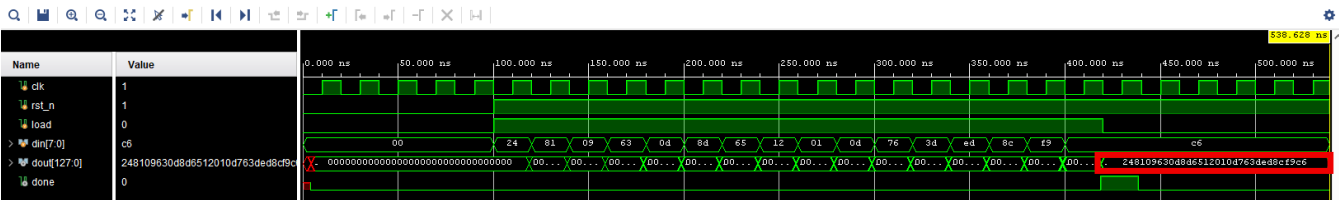
在模擬方面，我使用 \$random 的寫法，讓 din 隨機送進一個 byte，在連續送進 16 的 byte 之後，就可以得到 dout[127:0] 的完整資料，底下的模擬例子 dout[127:0]

的輸出結果為 128'h24_81_09_63_0d_65_12_01_0d_76_3d_ed_8c_f9_c6，那這 16 組 byte 就做為後續排序用。

Behavior Simulation :



Post-Route Simulation :



2. Output Module (PISO_16x8)

接著，這個 PISO_16x8 模組作為輸出端的設計，功能也與 design 時的描述一致，是將排序完成後的 128 bits 資料依序輸出，每次輸出 8 bits，連續輸出 16 次後即可完成整體資料的傳遞。

在 verilog 的寫法上，我也是利用 load 控制訊號與 4 bits 的計數器 count 來完成 16 次的輸出，但在輸出端因為已經有 128 bits 排序好的資料了，所以這裡就不需要再用 shift register 的方式傳遞，直接透過 index 取出對應的 8 bits 輸出到 dout，並在輸出最後一組資料時，將 done 訊號拉高一個 clock 表示結束。

Verilog :

```
1  `timescale 1ns / 1ps
2  // Company:
3  // Engineer:
4  //
5  // Create Date: 2025/11/08 16:38:46
6  // Design Name:
7  // Module Name: PISO_16x8
8  // Project Name:
9  // Target Devices:
10 // Tool Versions:
11 // Description:
12 //
13 // Dependencies:
14 //
15 // Revision:
16 // Revision 0.01 - File Created
17 // Additional Comments:
18 //
19 //
20 //
21
22
23 module PISO_16x8(
24     input wire clk,
25     input wire rst_n,
26     input wire load,
27     input wire [127:0] din,
28     output reg [7:0] dout,
29     output reg done
30 );
31
32     reg [3:0] count;
33     always @(posedge clk or negedge rst_n)begin
34         if(!rst_n)begin
35             dout <= 8'b0;
36             done <= 1'b0;
37             count <= 4'b0;
38         end
39         else begin
40             done <= 1'b0;
41             if(load)begin
42                 dout <= din[8*count+: 8];
43                 count <= count +1;
44                 if(count == 4'd15)begin
45                     done <= 1'b1;
46                 end
47             end
48         end
49     end
50 endmodule
```

Testbench :

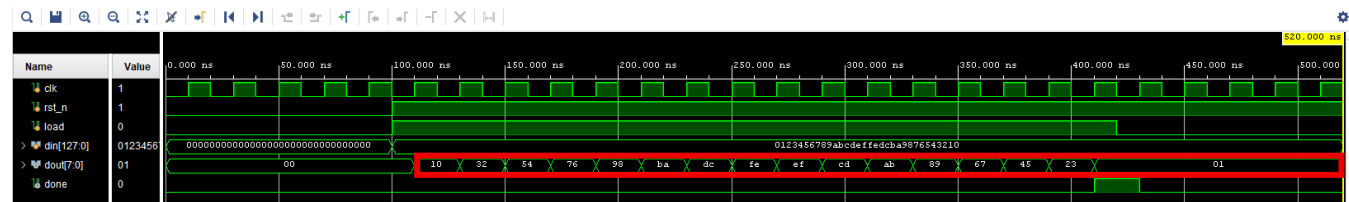
```
1  `timescale 1ns / 1ps
2  //////////////////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 2025/11/08 17:14:45
7  // Design Name:
8  // Module Name: tb_PISO_16x8
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 //////////////////////////////////////////////////
21
22
23 module tb_PISO_16x8;
24     reg clk;
25     reg rst_n;
26     reg load;
27     reg [127:0] din;
28     wire [7:0] dout;
29     wire done;
30
31     PISO_16x8 uut (
32         .clk(clk),
33         .rst_n(rst_n),
34         .load(load),
35         .din(din),
36         .dout(dout),
37         .done(done)
38     );
39
40     always #10 clk = ~clk;
41     initial begin
42         clk = 0;
43         rst_n = 0;
44         load = 0;
45         din = 128'd0;
46
47         #100;
48         rst_n = 1;
49         din = 128'h0123456789abcdeffedcba9876543210;
50
51         repeat (16) begin
52             @(negedge clk);
53             load = 1;
54         end
55
56         @(negedge clk);
57         load = 0;
58
59         #100;
60         $finish;
61     end
62 endmodule
```

Simulation :

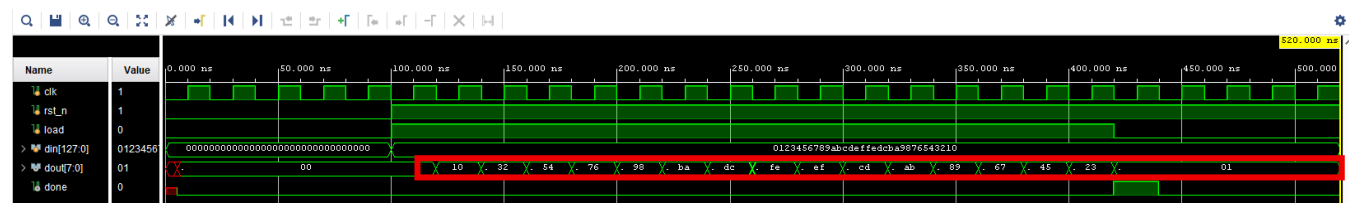
在模擬時，我設定 $din = 128'h0123456789abcdeffedcba9876543210$ ，接著經過

16 個 clock cycle 依序把 8 bits 資料傳出來到 dout[7:0]，所以可以看到 dout[7:0] 依序為 10, 32, 54, 76, 98, ba, dc, fe, ef, cd, ab, 89, 67, 45, 23, 01，並且在傳完 01 後，done 訊號拉高一個 clock cycle。

Behavior Simulation：



Post-Route Simulation：



3. 8 bits Compare Swap Unit

這個模組是整個 sorting network 的基本運算單元，主要負責兩個 8-bit 資料之間的大小比較與交換。每個 Compare Swap Unit 在 clock 上緣時執行比較運算，並根據輸入值的大小輸出對應的最小值 (min) 與最大值 (max)。

在 verilog 的實作上，模組包含兩個 8 bits 的 input 資料 A 與 B，以及對應的 output min 與 max，並且在 enable 訊號為高時啟動比較操作。當 $A \geq B$ 時，將最大值設為 A、最小值設為 B； $A < B$ 時，將最大值設為 B、最小值設為 A。

以硬體來說，Compare Swap Unit 邏輯等效於一個 8bits 比較器加上兩個 8bits 多工器，可在一個 clock 週期內完成比較與交換動作。這個基本單元會被重複使用於各層 Bitonic Merger 中，負責執行基本的比較交換，以下為我寫的 verilog 程式碼。

Verilog :

```
1 `timescale 1ns / 1ps
2 ///////////////////////////////////////////////////////////////////
3 // Company:
4 // Engineer:
5 //
6 // Create Date: 2025/11/07 15:12:46
7 // Design Name:
8 // Module Name: compare_swap_unit_8bits
9 // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21
22
23 module compare_swap_unit_8bits(
24     input wire [7:0] A,
25     input wire [7:0] B,
26     input wire clk,
27     input wire rst_n,
28     input wire enable,
29     output reg [7:0] min,
30     output reg [7:0] max
31 );
32
33 always @(posedge clk or negedge rst_n)begin
34     if (!rst_n)begin
35         min <= 8'b0;
36         max <= 8'b0;
37     end
38     else if (enable)begin
39         if (A>=B)begin
40             min <= B;
41             max <= A;
42         end
43         else begin
44             min <= A;
45             max <= B;
46         end
47     end
48 end
49 endmodule
```

Testbench :

```
1 `timescale 1ns / 1ps
2 ///////////////////////////////////////////////////////////////////
3 // Company:
4 // Engineer:
5 //
6 // Create Date: 2025/11/07 15:41:50
7 // Design Name:
8 // Module Name: tb_compare_swap_unit_8bits
9 // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
```



```

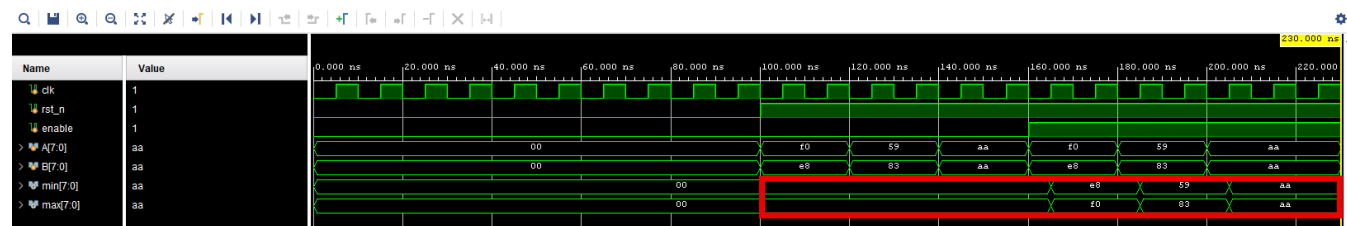
23 module tb_compare_swap_unit_8bits;
24     reg      clk;
25     reg      rst_n;
26     reg      enable;
27     reg [7:0] A, B;
28     wire [7:0] min, max;
29
30     compare_swap_unit_8bits dut (
31         .A(A),
32         .B(B),
33         .clk(clk),
34         .rst_n(rst_n),
35         .enable(enable),
36         .min(min),
37         .max(max)
38     );
39
40     always #5 clk = ~clk;
41     initial begin
42         clk = 0;
43         rst_n = 0;
44         A = 0;
45         B = 0;
46         enable = 0;
47
48         #100 rst_n = 1; enable = 0;
49
50         // === 測試 1: A > B ===
51         @(negedge clk);
52         A = 8'hf0; B = 8'he8;
53         #20
54
55         // === 測試 2: A < B ===
56         @(negedge clk);
57         A = 8'h59; B = 8'h83;
58         #20
59
60         // === 測試 3: A == B ===
61         @(negedge clk);
62         A = 8'haa; B = 8'haa;
63         #20
64
65         // === 測試 enable
66         enable = 1;
67
68         @(negedge clk);
69         A = 8'hf0; B = 8'he8;
70         #20
71
72         @(negedge clk);
73         A = 8'h59; B = 8'h83;
74         #20
75
76         @(negedge clk);
77         A = 8'haa; B = 8'haa;
78
79
80         #30;
81         $finish;
82     end
83 endmodule

```

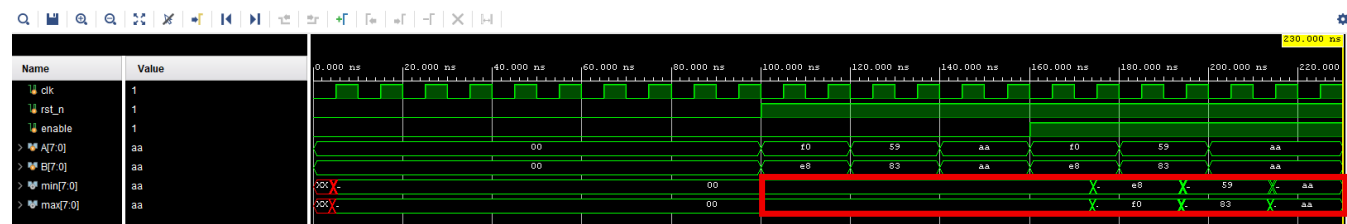
Simulation :

在模擬方面，我做了 6 組測試，前三組資料分別是在 enable 為 0 的情況下 $A > B$ 、 $A < B$ 、 $A = B$ ，後三組則是 enable 為 1 的情況 $A > B$ 、 $A < B$ 、 $A = B$ 的測試，如此就可以確認 compare swap 與 enable 的功能正不正常，結果可以發現在 enable 為 1 時，min 與 max 的輸出功能都是正確的。

Behavior Simulation :



Post-Route Simulation :



4. Bitonic Merger 4 inputs

接著，有了 Compare Swap Unit 之後，就可以來組 Bitonic Merger 4 inputs 的模組，他負責將 2 組 (上下兩部分) 已排序的 inputs 資料進行合併成為 4 個排序完成的輸出。整體架構分為兩層運算：

第一層執行 in_0 與 in_3、in_1 與 in_2 的資料進行大小比較，輸出的 min 與 max 先暫存在中間的 net 裡面。第二層再將中間結果 net[0] 與 net[1]、net[2] 與 net[3] 進行比較交換，完成最終的 4 組輸出 out_0 ~ out_3。

在 verilog 的寫法上，我利用 structure 的描述來完成小模組的合併，所以與 Compare Swap Unit 相同，Bitonic Merger 4 inputs 也為上緣觸發且由 enable 控制的模組，以下為我寫的 verilog 程式碼。

Verilog :

```
1 `timescale 1ns / 1ps
2 // Company:
3 // Engineer:
4 //
5 // Create Date: 2025/11/08 13:15:08
6 // Design Name:
7 // Module Name: bitonic_merger_4
8 // Project Name:
9 // Target Devices:
```

```

11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////
21
22
23 module bitonic_merger_4(
24     input wire [7:0] in_0,
25     input wire [7:0] in_1,
26     input wire [7:0] in_2,
27     input wire [7:0] in_3,
28     input wire clk,
29     input wire rst_n,
30     input wire enable,
31     output wire [7:0] out_0,
32     output wire [7:0] out_1,
33     output wire [7:0] out_2,
34     output wire [7:0] out_3
35 );
36
37     wire [7:0] net [0:3];
38
39     compare_swap_unit_8bits c0(.A(in_0),
40                               .B(in_3),
41                               .clk(clk),
42                               .rst_n(rst_n),
43                               .enable(enable),
44                               .min(net[0]),
45                               .max(net[3])
46                               );
47
48     compare_swap_unit_8bits c1(.A(in_1),
49                               .B(in_2),
50                               .clk(clk),
51                               .rst_n(rst_n),
52                               .enable(enable),
53                               .min(net[1]),
54                               .max(net[2])
55                               );
56
57     compare_swap_unit_8bits c2(.A(net[0]),
58                               .B(net[1]),
59                               .clk(clk),
60                               .rst_n(rst_n),
61                               .enable(enable),
62                               .min(out_0),
63                               .max(out_1)
64                               );
65
66     compare_swap_unit_8bits c3(.A(net[2]),
67                               .B(net[3]),
68                               .clk(clk),
69                               .rst_n(rst_n),
70                               .enable(enable),
71                               .min(out_2),
72                               .max(out_3)
73                               );
74
75 endmodule

```

Testbench :

```

1  `timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 2025/11/08 13:53:22
7  // Design Name:
8  // Module Name: tb_bitonic_merger_4
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21
22
23 module tb_bitonic_merger_4;
24     reg        clk;
25     reg        rst_n;
26     reg        enable;
27     reg [7:0]  in_0, in_1, in_2, in_3;
28     wire [7:0] out_0, out_1, out_2, out_3;
29
30     bitonic_merger_4 dut (
31         .in_0(in_0),
32         .in_1(in_1),
33         .in_2(in_2),
34         .in_3(in_3),
35         .clk(clk),
36         .rst_n(rst_n),
37         .enable(enable),
38         .out_0(out_0),
39         .out_1(out_1),
40         .out_2(out_2),
41         .out_3(out_3)
42     );
43
44     always #5 clk = ~clk;
45     initial begin
46         clk = 0;
47         rst_n = 0;
48         enable = 0;
49         in_0 = 0;
50         in_1 = 0;
51         in_2 = 0;
52         in_3 = 0;
53
54         #100;
55         rst_n = 1;
56         @(negedge clk);
57
58         // 測試1
59         enable = 1'b1;
60         in_0 = 8'h10;
61         in_1 = 8'h40;
62         in_2 = 8'h20;
63         in_3 = 8'h30;
64
65         // 等兩個週期經過 CS
66         repeat(2) @(negedge clk);
67
68         //測試2
69         enable = 1'b0;
70         in_0 = 8'hbb;
71         in_1 = 8'hff;
72         in_2 = 8'haa;
73         in_3 = 8'hcc;
74
75         @(negedge clk);
76         enable = 1'b1;

```

```

83         #20;
84         $finish;
85     end
86 endmodule

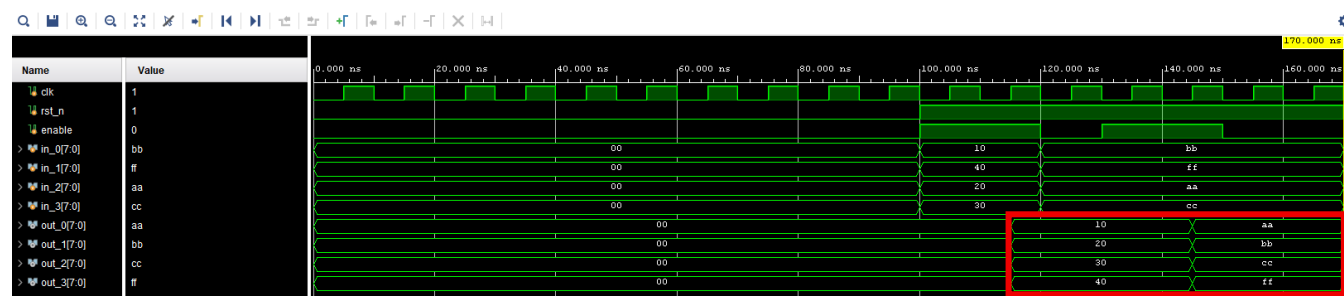
```

Simulation :

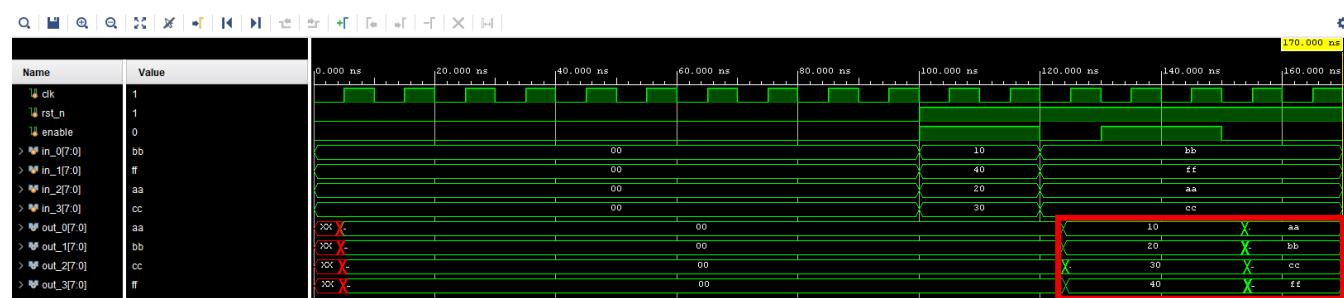
在模擬方面，我做了 2 組測試，由於 Bitonic Merger 的輸入資料一定要為 2 組已排序的資料才能正確合併輸出結果，所以我的輸入資料分別為：第一組 $in_0 = 8'h10$ 、 $in_1 = 8'h40$ 、 $in_2 = 8'h20$ 、 $in_3 = 8'h30$ ，第二組 $in_0 = 8'hbb$ 、 $in_1 = 8'hff$ 、 $in_2 = 8'haa$ 、 $in_3 = 8'hcc$ 。

其中 in_0 與 in_1 為已經排序好的 ($in_0 < in_1$)； in_2 與 in_3 也為已經排序好的 ($in_2 < in_3$)，選擇好輸入資料後，分別經過 2 個 clock cycle，最終輸出結果 $out_0 \sim out_3$ 分別為排序好的 $8'h10$ 、 $8'h20$ 、 $8'h30$ 、 $8'h40$ 和 $8'haa$ 、 $8'hbb$ 、 $8'hcc$ 、 $8'hff$ ，如此確認功能正確。

Behavior Simulation :



Post-Route Simulation :



5. Bitonic Merger 8 inputs

接著，來組 Bitonic Merger 8 inputs 的模組，他也是負責將 2 組 (上下兩部分) 已排序的 inputs 資料進行合併成為 8 個排序完成的輸出。整體架構分為三層運算，第一層運算後先將結果暫存在 net_0 內，第二層運算後則將結果暫存在 net_1 內，到第三層即可完成最終的 8 組輸出 out_0 ~ out_7。

在 verilog 的寫法上，我也是利用 structure 的描述來完成合併，以下為我寫的 verilog 程式碼。

Verilog :

```
1  `timescale 1ns / 1ps
2  // Company:
3  // Engineer:
4  //
5  // Create Date: 2025/11/08 14:20:41
6  // Design Name:
7  // Module Name: bitonic_merger_8
8  // Project Name:
9  // Target Devices:
10 // Tool Versions:
11 // Description:
12 //
13 // Dependencies:
14 //
15 // Revision:
16 // Revision 0.01 - File Created
17 // Additional Comments:
18 //
19 //
20 //
21
22
23 module bitonic_merger_8(
24     input wire [7:0] in_0,
25     input wire [7:0] in_1,
26     input wire [7:0] in_2,
27     input wire [7:0] in_3,
28     input wire [7:0] in_4,
29     input wire [7:0] in_5,
30     input wire [7:0] in_6,
31     input wire [7:0] in_7,
32     input wire clk,
33     input wire rst_n,
34     input wire enable,
35     output wire [7:0] out_0,
36     output wire [7:0] out_1,
37     output wire [7:0] out_2,
38     output wire [7:0] out_3,
39     output wire [7:0] out_4,
40     output wire [7:0] out_5,
41     output wire [7:0] out_6,
42     output wire [7:0] out_7
43 );
44
45     wire [7:0] net_0 [0:7];
46     wire [7:0] net_1 [0:7];
47
48     compare_swap_unit_8bits c0(.A(in_0), .B(in_7), .clk(clk), .rst_n(rst_n), .enable(enable), .min(net_0[0]), .max(net_0[7]));
49     compare_swap_unit_8bits c1(.A(in_1), .B(in_6), .clk(clk), .rst_n(rst_n), .enable(enable), .min(net_0[1]), .max(net_0[6]));
50     compare_swap_unit_8bits c2(.A(in_2), .B(in_5), .clk(clk), .rst_n(rst_n), .enable(enable), .min(net_0[2]), .max(net_0[5]));
51     compare_swap_unit_8bits c3(.A(in_3), .B(in_4), .clk(clk), .rst_n(rst_n), .enable(enable), .min(net_0[3]), .max(net_0[4]));
52
53     compare_swap_unit_8bits c4(.A(net_0[0]), .B(net_0[2]), .clk(clk), .rst_n(rst_n), .enable(enable), .min(net_1[0]), .max(net_1[2]));
54     compare_swap_unit_8bits c5(.A(net_0[1]), .B(net_0[3]), .clk(clk), .rst_n(rst_n), .enable(enable), .min(net_1[1]), .max(net_1[3]));
55     compare_swap_unit_8bits c6(.A(net_0[4]), .B(net_0[6]), .clk(clk), .rst_n(rst_n), .enable(enable), .min(net_1[4]), .max(net_1[6]));
56     compare_swap_unit_8bits c7(.A(net_0[5]), .B(net_0[7]), .clk(clk), .rst_n(rst_n), .enable(enable), .min(net_1[5]), .max(net_1[7]));
57
58     compare_swap_unit_8bits c8(.A(net_1[0]), .B(net_1[1]), .clk(clk), .rst_n(rst_n), .enable(enable), .min(out_0), .max(out_1));
59     compare_swap_unit_8bits c9(.A(net_1[2]), .B(net_1[3]), .clk(clk), .rst_n(rst_n), .enable(enable), .min(out_2), .max(out_3));
60     compare_swap_unit_8bits c10(.A(net_1[4]), .B(net_1[5]), .clk(clk), .rst_n(rst_n), .enable(enable), .min(out_4), .max(out_5));
61     compare_swap_unit_8bits c11(.A(net_1[6]), .B(net_1[7]), .clk(clk), .rst_n(rst_n), .enable(enable), .min(out_6), .max(out_7));
62
63 endmodule
```

Testbench :

```
1  `timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 2025/11/08 14:32:31
7  // Design Name:
8  // Module Name: tb_bitonic_merger_8
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21
22
23 module tb_bitonic_merger_8;
24     reg        clk;
25     reg        rst_n;
26     reg        enable;
27     reg [7:0]  in_0, in_1, in_2, in_3, in_4, in_5, in_6, in_7;
28     wire [7:0] out_0, out_1, out_2, out_3, out_4, out_5, out_6, out_7;
29
30     bitonic_merger_8 dut (
31         .in_0(in_0),
32         .in_1(in_1),
33         .in_2(in_2),
34         .in_3(in_3),
35         .in_4(in_4),
36         .in_5(in_5),
37         .in_6(in_6),
38         .in_7(in_7),
39         .clk(clk),
40         .rst_n(rst_n),
41         .enable(enable),
42         .out_0(out_0),
43         .out_1(out_1),
44         .out_2(out_2),
45         .out_3(out_3),
46         .out_4(out_4),
47         .out_5(out_5),
48         .out_6(out_6),
49         .out_7(out_7)
50     );
51
52     always #5 clk = ~clk;
53     initial begin
54         clk      = 0;
55         rst_n    = 0;
56         enable   = 0;
57         in_0     = 0;
58         in_1     = 0;
59         in_2     = 0;
60         in_3     = 0;
61         in_4     = 0;
62         in_5     = 0;
63         in_6     = 0;
64         in_7     = 0;
65
66         #100;
67         rst_n = 1;
68         @(negedge clk);
69
70         // 測試1
71         enable = 1'b1;
72         in_0 = 8'h12;
73         in_1 = 8'h34;
74         in_2 = 8'h67;
75         in_3 = 8'h88;
```

```

76     in_4 = 8'h09;
77     in_5 = 8'h45;
78     in_6 = 8'h57;
79     in_7 = 8'hab;
80
81     // 等三個週期經過 cs
82     repeat(3) @(negedge clk);
83
84     //測試2
85     enable = 1'b0;
86     in_0 = 8'h11;
87     in_1 = 8'h44;
88     in_2 = 8'h88;
89     in_3 = 8'hcc;
90     in_4 = 8'h33;
91     in_5 = 8'h66;
92     in_6 = 8'h99;
93     in_7 = 8'hff;
94
95     @(negedge clk);
96     enable = 1'b1;
97
98     // 等三個週期經過 cs
99     repeat(3) @(negedge clk);
100    enable = 1'b0;
101    #20;
102    $finish;
103 end
104 endmodule

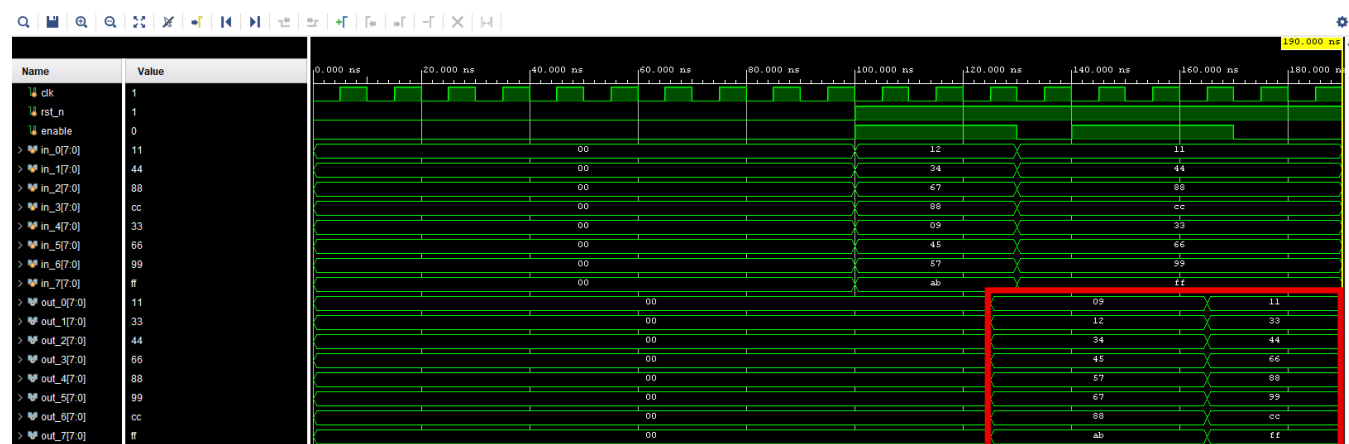
```

Simulation :

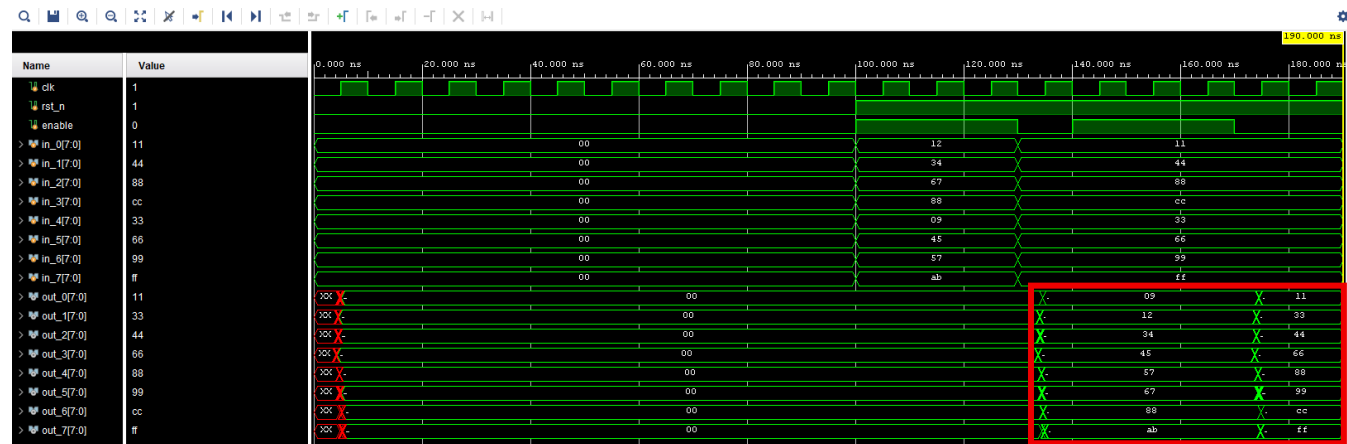
在模擬方面，我也做了 2 組測試，也因為剛才提到的 Bitonic Merger 的輸入資料一定要為 2 組已排序的資料，所以我的輸入資料分別為：第一組 in_0=8'h12、in_1=8'h34、in_2=8'h67、in_3=8'h88、in_4=8'h09、in_5=8'h45、in_6=8'h57、in_7=8'hab，第二組 in_0=8'h11、in_1=8'h44、in_2=8'h88、in_3=8'hcc、in_4=8'h33、in_5=8'h66、in_6=8'h99、in_7=8'hff。

選擇好輸入資料後，分別經過 3 個 clock cycle，最終輸出結果 out_0~out_7 分別為排序好的 8'h09、8'h12、8'h34、8'h45、8'h57、8'h67、8'h88、8'hab 和 8'h11、8'h33、8'h44、8'h66、8'h88、8'h99、8'hcc、8'hff，如此確認功能正確。

Behavior Simulation :



Post-Route Simulation :



6. Bitonic Merger 16 inputs

接著，來組 Bitonic Merger 16 inputs 的模組，與之前相同，他也是負責將 2 組 (上下兩部分) 已排序的 inputs 資料進行合併成為 16 個排序完成的輸出。整體架構分為四層運算，第一層運算後先將結果暫存在 net_0 內，第二層運算後將結果暫存在 net_1 內，第三層將結果暫存在 net_2 內，最後第四層即可完成最終的 8 組輸出 out_0 ~ out_15。

在 verilog 的寫法上，也是利用 structure 的描述來完成合併，以下為我寫的 verilog 程式碼。

Verilog :

```

7 // Design Name:
8 // Module Name: bitonic_merger_16
9 // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21
22
23 module bitonic_merger_16(
24     input wire [7:0] in_0,
25     input wire [7:0] in_1,
26     input wire [7:0] in_2,
27     input wire [7:0] in_3,
28     input wire [7:0] in_4,
29     input wire [7:0] in_5,
30     input wire [7:0] in_6,
31     input wire [7:0] in_7,
32     input wire [7:0] in_8,
33     input wire [7:0] in_9,
34     input wire [7:0] in_10,

```

```

35     input wire [7:0] in_11,
36     input wire [7:0] in_12,
37     input wire [7:0] in_13,
38     input wire [7:0] in_14,
39     input wire [7:0] in_15,
40     input wire clk,
41     input wire rst_n,
42     input wire enable,
43     output wire [7:0] out_0,
44     output wire [7:0] out_1,
45     output wire [7:0] out_2,
46     output wire [7:0] out_3,
47     output wire [7:0] out_4,
48     output wire [7:0] out_5,
49     output wire [7:0] out_6,
50     output wire [7:0] out_7,
51     output wire [7:0] out_8,
52     output wire [7:0] out_9,
53     output wire [7:0] out_10,
54     output wire [7:0] out_11,
55     output wire [7:0] out_12,
56     output wire [7:0] out_13,
57     output wire [7:0] out_14,
58     output wire [7:0] out_15
59 );

61     wire [7:0] net_0 [0:15];
62     wire [7:0] net_1 [0:15];
63     wire [7:0] net_2 [0:15];
64
65     compare_swap_unit_8bits c0(.A(in_0), .B(in_15), .clk(clk), .rst_n(rst_n), .enable(enable), .min(net_0[0]), .max(net_0[15]));
66     compare_swap_unit_8bits c1(.A(in_1), .B(in_14), .clk(clk), .rst_n(rst_n), .enable(enable), .min(net_0[1]), .max(net_0[14]));
67     compare_swap_unit_8bits c2(.A(in_2), .B(in_13), .clk(clk), .rst_n(rst_n), .enable(enable), .min(net_0[2]), .max(net_0[13]));
68     compare_swap_unit_8bits c3(.A(in_3), .B(in_12), .clk(clk), .rst_n(rst_n), .enable(enable), .min(net_0[3]), .max(net_0[12]));
69     compare_swap_unit_8bits c4(.A(in_4), .B(in_11), .clk(clk), .rst_n(rst_n), .enable(enable), .min(net_0[4]), .max(net_0[11]));
70     compare_swap_unit_8bits c5(.A(in_5), .B(in_10), .clk(clk), .rst_n(rst_n), .enable(enable), .min(net_0[5]), .max(net_0[10]));
71     compare_swap_unit_8bits c6(.A(in_6), .B(in_9), .clk(clk), .rst_n(rst_n), .enable(enable), .min(net_0[6]), .max(net_0[9]));
72     compare_swap_unit_8bits c7(.A(in_7), .B(in_8), .clk(clk), .rst_n(rst_n), .enable(enable), .min(net_0[7]), .max(net_0[8]));
73
74     compare_swap_unit_8bits c8(.A(net_0[0]), .B(net_0[4]), .clk(clk), .rst_n(rst_n), .enable(enable), .min(net_1[0]), .max(net_1[4]));
75     compare_swap_unit_8bits c9(.A(net_0[1]), .B(net_0[5]), .clk(clk), .rst_n(rst_n), .enable(enable), .min(net_1[1]), .max(net_1[5]));
76     compare_swap_unit_8bits c10(.A(net_0[2]), .B(net_0[6]), .clk(clk), .rst_n(rst_n), .enable(enable), .min(net_1[2]), .max(net_1[6]));
77     compare_swap_unit_8bits c11(.A(net_0[3]), .B(net_0[7]), .clk(clk), .rst_n(rst_n), .enable(enable), .min(net_1[3]), .max(net_1[7]));
78     compare_swap_unit_8bits c12(.A(net_0[8]), .B(net_0[12]), .clk(clk), .rst_n(rst_n), .enable(enable), .min(net_1[8]), .max(net_1[12]));
79     compare_swap_unit_8bits c13(.A(net_0[9]), .B(net_0[13]), .clk(clk), .rst_n(rst_n), .enable(enable), .min(net_1[9]), .max(net_1[13]));
80     compare_swap_unit_8bits c14(.A(net_0[10]), .B(net_0[14]), .clk(clk), .rst_n(rst_n), .enable(enable), .min(net_1[10]), .max(net_1[14]));
81     compare_swap_unit_8bits c15(.A(net_0[11]), .B(net_0[15]), .clk(clk), .rst_n(rst_n), .enable(enable), .min(net_1[11]), .max(net_1[15]));
82
83     compare_swap_unit_8bits c16(.A(net_1[0]), .B(net_1[2]), .clk(clk), .rst_n(rst_n), .enable(enable), .min(net_2[0]), .max(net_2[2]));
84     compare_swap_unit_8bits c17(.A(net_1[1]), .B(net_1[3]), .clk(clk), .rst_n(rst_n), .enable(enable), .min(net_2[1]), .max(net_2[3]));
85     compare_swap_unit_8bits c18(.A(net_1[4]), .B(net_1[6]), .clk(clk), .rst_n(rst_n), .enable(enable), .min(net_2[4]), .max(net_2[6]));
86     compare_swap_unit_8bits c19(.A(net_1[5]), .B(net_1[7]), .clk(clk), .rst_n(rst_n), .enable(enable), .min(net_2[5]), .max(net_2[7]));
87     compare_swap_unit_8bits c20(.A(net_1[8]), .B(net_1[10]), .clk(clk), .rst_n(rst_n), .enable(enable), .min(net_2[8]), .max(net_2[10]));
88     compare_swap_unit_8bits c21(.A(net_1[9]), .B(net_1[11]), .clk(clk), .rst_n(rst_n), .enable(enable), .min(net_2[9]), .max(net_2[11]));
89     compare_swap_unit_8bits c22(.A(net_1[12]), .B(net_1[14]), .clk(clk), .rst_n(rst_n), .enable(enable), .min(net_2[12]), .max(net_2[14]));
90     compare_swap_unit_8bits c23(.A(net_1[13]), .B(net_1[15]), .clk(clk), .rst_n(rst_n), .enable(enable), .min(net_2[13]), .max(net_2[15]));
91
92     compare_swap_unit_8bits c24(.A(net_2[0]), .B(net_2[1]), .clk(clk), .rst_n(rst_n), .enable(enable), .min(out_0), .max(out_1));
93     compare_swap_unit_8bits c25(.A(net_2[2]), .B(net_2[3]), .clk(clk), .rst_n(rst_n), .enable(enable), .min(out_2), .max(out_3));
94     compare_swap_unit_8bits c26(.A(net_2[4]), .B(net_2[5]), .clk(clk), .rst_n(rst_n), .enable(enable), .min(out_4), .max(out_5));
95     compare_swap_unit_8bits c27(.A(net_2[6]), .B(net_2[7]), .clk(clk), .rst_n(rst_n), .enable(enable), .min(out_6), .max(out_7));
96     compare_swap_unit_8bits c28(.A(net_2[8]), .B(net_2[9]), .clk(clk), .rst_n(rst_n), .enable(enable), .min(out_8), .max(out_9));
97     compare_swap_unit_8bits c29(.A(net_2[10]), .B(net_2[11]), .clk(clk), .rst_n(rst_n), .enable(enable), .min(out_10), .max(out_11));
98     compare_swap_unit_8bits c30(.A(net_2[12]), .B(net_2[13]), .clk(clk), .rst_n(rst_n), .enable(enable), .min(out_12), .max(out_13));
99     compare_swap_unit_8bits c31(.A(net_2[14]), .B(net_2[15]), .clk(clk), .rst_n(rst_n), .enable(enable), .min(out_14), .max(out_15));
100 endmodule

```

Testbench :

```

16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21
22
23 module tb_bitonic_merger_16;
24     reg clk;
25     reg rst_n;
26     reg enable;
27     reg [7:0] in_0, in_1, in_2, in_3, in_4, in_5, in_6, in_7, in_8, in_9, in_10, in_11, in_12, in_13, in_14, in_15;
28     wire [7:0] out_0, out_1, out_2, out_3, out_4, out_5, out_6, out_7, out_8, out_9, out_10, out_11, out_12, out_13, out_14, out_15;
29
30     bitonic_merger_16 dut (
31         .in_0(in_0),
32         .in_1(in_1),
33         .in_2(in_2),
34         .in_3(in_3),
35         .in_4(in_4),
36         .in_5(in_5),

```

```

37     .in_6(in_6),
38     .in_7(in_7),
39     .in_8(in_8),
40     .in_9(in_9),
41     .in_10(in_10),
42     .in_11(in_11),
43     .in_12(in_12),
44     .in_13(in_13),
45     .in_14(in_14),
46     .in_15(in_15),
47     .clk(clk),
48     .rst_n(rst_n),
49     .enable(enable),
50     .out_0(out_0),
51     .out_1(out_1),
52     .out_2(out_2),
53     .out_3(out_3),
54     .out_4(out_4),
55     .out_5(out_5),
56     .out_6(out_6),
57     .out_7(out_7),
58     .out_8(out_8),
59     .out_9(out_9),
60     .out_10(out_10),
61     .out_11(out_11),
62     .out_12(out_12),
63     .out_13(out_13),
64     .out_14(out_14),
65     .out_15(out_15)
66 );

68 always #5 clk = ~clk;
69 initial begin
70     clk = 0;
71     rst_n = 0;
72     enable = 0;
73     in_0 = 0;
74     in_1 = 0;
75     in_2 = 0;
76     in_3 = 0;
77     in_4 = 0;
78     in_5 = 0;
79     in_6 = 0;
80     in_7 = 0;
81     in_8 = 0;
82     in_9 = 0;
83     in_10 = 0;
84     in_11 = 0;
85     in_12 = 0;
86     in_13 = 0;
87     in_14 = 0;
88     in_15 = 0;
89
90     #100;
91     rst_n = 1;
92     @(negedge clk);
93
94     // 測試1
95     enable = 1'b1;
96     in_0 = 8'h02;
97     in_1 = 8'h24;
98     in_2 = 8'h57;
99     in_3 = 8'h98;
100    in_4 = 8'ha7;
101    in_5 = 8'hc5;
102    in_6 = 8'hea;
103    in_7 = 8'hf9;
104    in_8 = 8'h34;
105    in_9 = 8'h46;
106    in_10 = 8'h78;
107    in_11 = 8'h94;
108    in_12 = 8'hb1;
109    in_13 = 8'hc9;
110    in_14 = 8'hda;
111    in_15 = 8'he9;
112

```

```

113 // 等四個週期經過 cs
114 repeat(4) @(negedge clk);
115
116 //測試2
117 enable = 1'b0;
118 in_0 = 8'h08;
119 in_1 = 8'h29;
120 in_2 = 8'h58;
121 in_3 = 8'h66;
122 in_4 = 8'h7e;
123 in_5 = 8'h9a;
124 in_6 = 8'ha7;
125 in_7 = 8'hcl;
126 in_8 = 8'h07;
127 in_9 = 8'h19;
128 in_10 = 8'h38;
129 in_11 = 8'h77;
130 in_12 = 8'h99;
131 in_13 = 8'ha6;
132 in_14 = 8'hb7;
133 in_15 = 8'he5;
134
135 @(negedge clk);
136 enable = 1'b1;
137
138 // 等四個週期經過 cs
139 repeat(4) @(negedge clk);
140 enable = 1'b0;
141
142 #20;
143 $finish;
144 end
145 endmodule

```

Simulation :

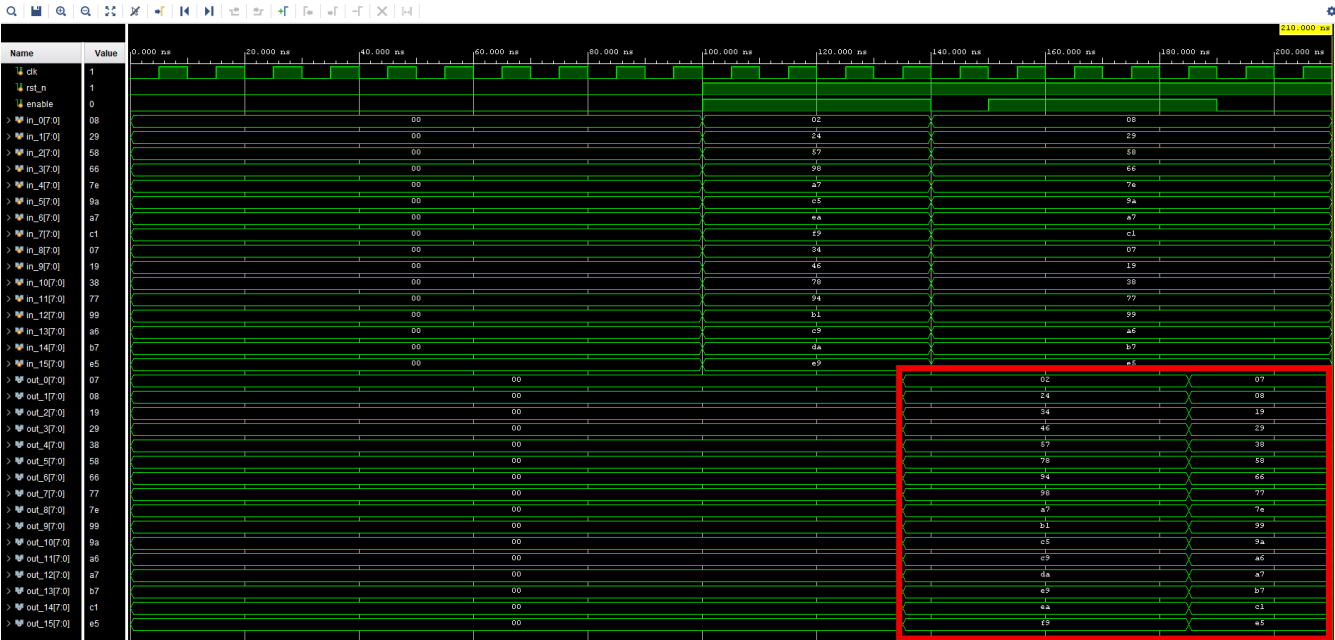
在模擬方面，也做了 2 組測試，這次的輸入資料為：

第一組的 in_0~in15 分別為 8'h02、8'h24、8'h57、8'h98、8'ha7、8'hc5、8'hea、8'hf9、8'h34、8'h46、8'h78、8'h94、8'hbe、8'hc9、8'hda、8'he9

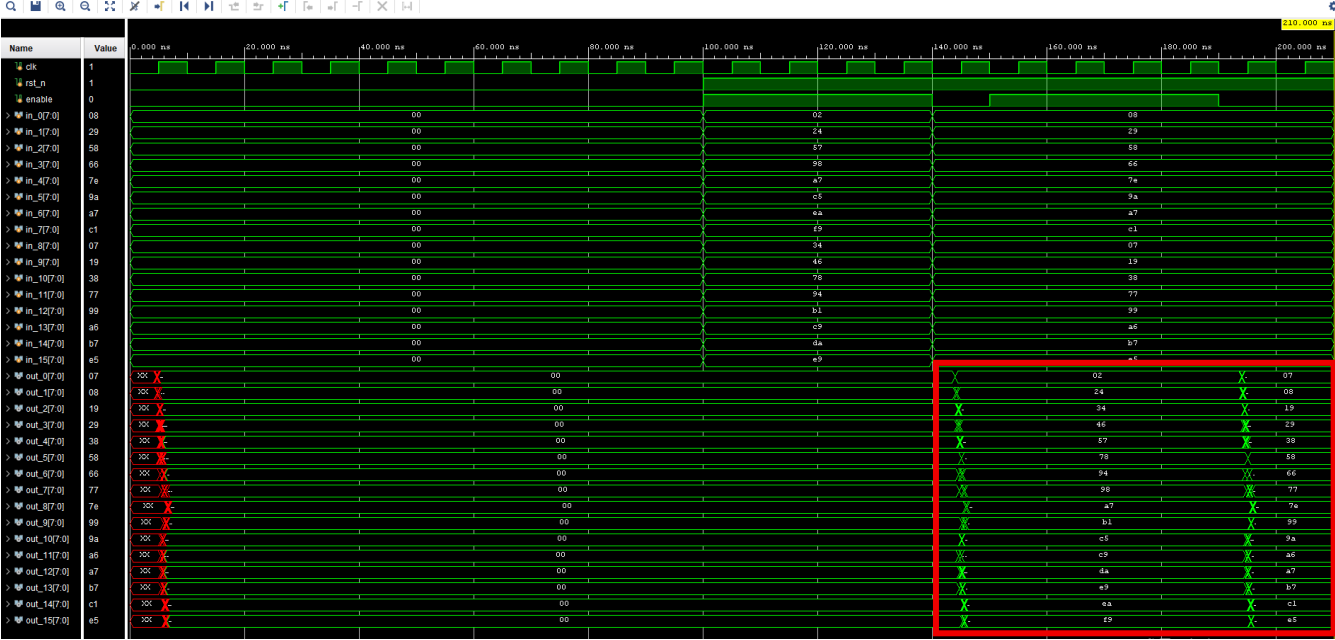
第二組的 in_0~in15 分別為 8'h08、8'h29、8'h58、8'h66、8'h7e、8'h9a、8'ha7、8'hcl、8'h07、8'h19、8'h38、8'h77、8'h99、8'ha6、8'hb7、8'he5

選擇好輸入資料後，分別經過 4 個 clock cycle，最終輸出結果 out_0~out_15 分別為排序好的 8'h02、8'h24、8'h34、8'h46、8'h57、8'h78、8'h94、8'h98、8'ha7、8'hb1、8'hc5、8'hc9、8'hda、8'he9、8'hea、8'hf9 和 8'h07、8'h08、8'h19、8'h29、8'h38、8'h58、8'h66、8'h77、8'h7e、8'h99、8'h9a、8'ha6、8'ha7、8'hb7、8'hcl、8'he5，如此確認功能正確。

Behavior Simulation :



Post-Route Simulation :



7. Bitonic Sorter 16 inputs

接著，Bitonic Merger 都完成後就可以來組 Bitonic Sorter 16 inputs 了，跟之前 desgin 時的介紹相同，一個 Bitonic Sorter 16 inputs 可以由 1*(16 input Bitonic Merger) + 2*(8 input Bitonic Merger) + 4*(4 input Bitonic Merger) + 8*(2 input Bitonic Merger) 所組成。

在 verilog 的寫法上，直接利用 structure 的描述來完成之前子模組的合併，以下為我寫的 verilog 程式碼。

Verilog :

```
1  `timescale 1ns / 1ps
2  // Company:
3  // Engineer:
4  //
5  // Create Date: 2025/11/08 15:24:30
6  // Design Name:
7  // Module Name: bitonic_sorter_16
8  // Project Name:
9  // Target Devices:
10 // Tool Versions:
11 // Description:
12 //
13 // Dependencies:
14 //
15 // Revision:
16 // Revision 0.01 - File Created
17 // Additional Comments:
18 //
19 //
20 //
21
22
23 module bitonic_sorter_16(
24     input wire [127:0] in,
25     input wire clk,
26     input wire rst_n,
27     input wire enable,
28     output wire [127:0] out
29 );
30
31     wire [7:0] net_0 [0:15];
32     wire [7:0] net_1 [0:15];
33     wire [7:0] net_2 [0:15];
34
35     //stage1
36     compare_swap_unit_8bits c0(.A(in[7:0] ), .B(in[15:8] ), .clk(clk), .rst_n(rst_n), .enable(enable), .min(net_0[0] ), .max(net_0[1] ));
37     compare_swap_unit_8bits c1(.A(in[23:16] ), .B(in[31:24] ), .clk(clk), .rst_n(rst_n), .enable(enable), .min(net_0[2] ), .max(net_0[3] ));
38     compare_swap_unit_8bits c2(.A(in[39:32] ), .B(in[47:40] ), .clk(clk), .rst_n(rst_n), .enable(enable), .min(net_0[4] ), .max(net_0[5] ));
39     compare_swap_unit_8bits c3(.A(in[55:48] ), .B(in[63:56] ), .clk(clk), .rst_n(rst_n), .enable(enable), .min(net_0[6] ), .max(net_0[7] ));
40     compare_swap_unit_8bits c4(.A(in[71:64] ), .B(in[79:72] ), .clk(clk), .rst_n(rst_n), .enable(enable), .min(net_0[8] ), .max(net_0[9] ));
41     compare_swap_unit_8bits c5(.A(in[87:80] ), .B(in[95:88] ), .clk(clk), .rst_n(rst_n), .enable(enable), .min(net_0[10] ), .max(net_0[11] ));
42     compare_swap_unit_8bits c6(.A(in[103:96] ), .B(in[111:104] ), .clk(clk), .rst_n(rst_n), .enable(enable), .min(net_0[12] ), .max(net_0[13] ));
43     compare_swap_unit_8bits c7(.A(in[119:112] ), .B(in[127:120] ), .clk(clk), .rst_n(rst_n), .enable(enable), .min(net_0[14] ), .max(net_0[15] ));
44
45     //stage2~3
46     bitonic_merger_4 b4_0(.in_0(net_0[0]),
47         .in_1(net_0[1]),
48         .in_2(net_0[2]),
49         .in_3(net_0[3]),
50         .clk(clk),
51         .rst_n(rst_n),
52         .enable(enable),
53         .out_0(net_1[0]),
54         .out_1(net_1[1]),
55         .out_2(net_1[2]),
56         .out_3(net_1[3]));
57     bitonic_merger_4 b4_1(.in_0(net_0[4]),
58         .in_1(net_0[5]),
59         .in_2(net_0[6]),
60         .in_3(net_0[7]),
61         .clk(clk),
62         .rst_n(rst_n),
63         .enable(enable),
64         .out_0(net_1[4]),
65         .out_1(net_1[5]),
66         .out_2(net_1[6]),
67         .out_3(net_1[7]));
```

```

68 bitonic_merger_4 b4_2(.in_0(net_0[8]),
69                      .in_1(net_0[9]),
70                      .in_2(net_0[10]),
71                      .in_3(net_0[11]),
72                      .clk(clk),
73                      .rst_n(rst_n),
74                      .enable(enable),
75                      .out_0(net_1[8]),
76                      .out_1(net_1[9]),
77                      .out_2(net_1[10]),
78                      .out_3(net_1[11]));
79 bitonic_merger_4 b4_3(.in_0(net_0[12]),
80                      .in_1(net_0[13]),
81                      .in_2(net_0[14]),
82                      .in_3(net_0[15]),
83                      .clk(clk),
84                      .rst_n(rst_n),
85                      .enable(enable),
86                      .out_0(net_1[12]),
87                      .out_1(net_1[13]),
88                      .out_2(net_1[14]),
89                      .out_3(net_1[15]));
90

91 //stage4~6
92 bitonic_merger_8 b8_0(.in_0(net_1[0]),
93                      .in_1(net_1[1]),
94                      .in_2(net_1[2]),
95                      .in_3(net_1[3]),
96                      .in_4(net_1[4]),
97                      .in_5(net_1[5]),
98                      .in_6(net_1[6]),
99                      .in_7(net_1[7]),
100                     .clk(clk),
101                     .rst_n(rst_n),
102                     .enable(enable),
103                     .out_0(net_2[0]),
104                     .out_1(net_2[1]),
105                     .out_2(net_2[2]),
106                     .out_3(net_2[3]),
107                     .out_4(net_2[4]),
108                     .out_5(net_2[5]),
109                     .out_6(net_2[6]),
110                     .out_7(net_2[7]));
111 bitonic_merger_8 b8_1(.in_0(net_1[8]),
112                      .in_1(net_1[9]),
113                      .in_2(net_1[10]),
114                      .in_3(net_1[11]),
115                      .in_4(net_1[12]),
116                      .in_5(net_1[13]),
117                      .in_6(net_1[14]),
118                      .in_7(net_1[15]),
119                      .clk(clk),
120                      .rst_n(rst_n),
121                      .enable(enable),
122                      .out_0(net_2[8]),
123                      .out_1(net_2[9]),
124                      .out_2(net_2[10]),
125                      .out_3(net_2[11]),
126                      .out_4(net_2[12]),
127                      .out_5(net_2[13]),
128                      .out_6(net_2[14]),
129                      .out_7(net_2[15]));
130

131 //stage7~10
132 bitonic_merger_16 b16_0(.in_0(net_2[0]),
133                        .in_1(net_2[1]),
134                        .in_2(net_2[2]),
135                        .in_3(net_2[3]),
136                        .in_4(net_2[4]),
137                        .in_5(net_2[5]),
138                        .in_6(net_2[6]),
139                        .in_7(net_2[7]),
140                        .in_8(net_2[8]),
141                        .in_9(net_2[9]),
142                        .in_10(net_2[10]),
143                        .in_11(net_2[11]),
144                        .in_12(net_2[12]),
145                        .in_13(net_2[13]),
146                        .in_14(net_2[14]),
147                        .in_15(net_2[15]),
148                        .clk(clk),
149                        .rst_n(rst_n),
150                        .enable(enable),
151                        .out_0(out[7:0]),
152                        .out_1(out[15:8]),
153                        .out_2(out[23:16]),
154                        .out_3(out[31:24]),
155                        .out_4(out[39:32]),
156                        .out_5(out[47:40]),
157                        .out_6(out[55:48]),
158                        .out_7(out[63:56]),
159                        .out_8(out[71:64]),
160                        .out_9(out[79:72]),
161                        .out_10(out[87:80]),

```

```

162         .out_11(out[95:88]),
163         .out_12(out[103:96]),
164         .out_13(out[111:104]),
165         .out_14(out[119:112]),
166         .out_15(out[127:120]));
167
168     endmodule

```

Testbench :

```

1  `timescale 1ns / 1ps
2  //////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 2025/11/08 15:57:33
7  // Design Name:
8  // Module Name: tb_bitonic_sorter_16
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 //////////////////////////////////////
21
22
23 module tb_bitonic_sorter_16;
24     reg        clk;
25     reg        rst_n;
26     reg        enable;
27     reg [127:0] in;
28     wire [127:0] out;
29
30     bitonic_sorter_16 dut (
31         .in(in),
32         .clk(clk),
33         .rst_n(rst_n),
34         .enable(enable),
35         .out(out)
36     );
37
38     always #5 clk = ~clk;
39     initial begin
40         clk = 0;
41         rst_n = 0;
42         enable = 0;
43         in = 0;
44
45         #100;
46         rst_n = 1;
47         @(negedge clk);
48
49         // 測試1
50         enable = 1'b1;
51         in = 128'hcc_de_87_3b_1f_50_b6_7a_61_a4_98_01_45_ec_23_ff;
52
53         // 等十個週期經過 CS
54         repeat(10) @(negedge clk);
55
56         //測試2
57         enable = 1'b0;
58         in = 128'h02_f0_99_5f_ee_17_b9_a0_6c_d1_78_24_c6_8e_4a_3c;
59
60         @(negedge clk);
61         enable = 1'b1;
62
63         // 等十個週期經過 CS
64         repeat(10) @(negedge clk);
65         enable = 1'b0;
66
67         #100;
68         $finish;
69     end
70 endmodule

```


Simulation :

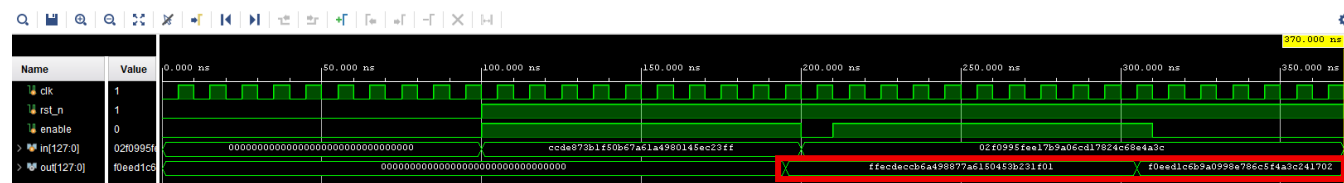
在模擬方面，也做了 2 組測試，但是與之前 Bitonic Merger 不同的是，這次完整的 Bitonic Sorter 的輸入資料並不用為事先排序好的兩組資料了！所以這次我的輸入資料可以分別為：

第一組 in = 128'hcc_de_87_3b_1f_50_b6_7a_61_a4_98_01_45_ec_23_ff

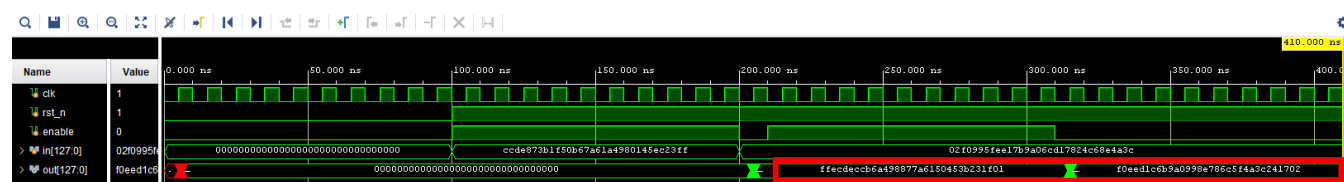
第二組 in = 128'h02_f0_99_5f_ee_17_b9_a0_6c_d1_78_24_c6_8e_4a_3c

選擇好輸入資料後，分別經過 4 個 clock cycle，最終輸出結果 out 分別為 128'hff_ec_de_cc_b6_a4_98_87_7a_61_50_45_3b_23_1f_01 與 128'hf0_ee_d1_c6_b9_a0_99_8e_78_6c_5f_4a_3c_24_17_02，如此確認功能正確。

Behavior Simulation :



Post-Route Simulation :



8. Datapath

接著，把三個之前的子模組 SIPO_16x8、Bitonic Sorter 16 inputs、PISO_16x8 合在一起成為最終的 datapath 就行了。在 verilog 的寫法上，都與之前相同，利用 structure 的描述來完成子模組的合併，以下為我寫的 verilog 程式碼。

Verilog :

```

1  `timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 2025/11/08 17:38:23
7  // Design Name:
8  // Module Name: datapath
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21 module datapath(
22     input wire clk,
23     input wire rst_n,
24     input wire load_in,
25     input wire load_out,
26     input wire enable_bs,
27     input wire [7:0] din,
28     output wire [7:0] dout,
29     output wire done
30 );
31     wire [127:0] net0;
32     wire [127:0] net1;
33     SIFO_16x8 s0(.clk(clk),
34                 .rst_n(rst_n),
35                 .load(load_in),
36                 .din(din),
37                 .dout(net0),
38                 .done()
39 );
40     bitonic_sorter_16 b0(.clk(clk),
41                         .rst_n(rst_n),
42                         .enable(enable_bs),
43                         .in(net0),
44                         .out(net1)
45 );
46     PISO_16x8 p0(.clk(clk),
47                 .rst_n(rst_n),
48                 .load(load_out),
49                 .din(net1),
50                 .dout(dout),
51                 .done(done)
52 );
53 endmodule

```

Testbench :

```

1  `timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 2025/11/09 14:11:44
7  // Design Name:
8  // Module Name: tb_datapath
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //

```

```

23 module tb_datapath;
24     reg        clk;
25     reg        rst_n;
26     reg        load_in;
27     reg        load_out;
28     reg        enable_bs;
29     reg [7:0]  din;
30     wire [7:0] dout;
31     wire        done;
32
33     datapath dut (
34         .clk(clk),
35         .rst_n(rst_n),
36         .load_in(load_in),
37         .load_out(load_out),
38         .enable_bs(enable_bs),
39         .din(din),
40         .dout(dout),
41         .done(done)
42     );
43
44     always #10 clk = ~clk;
45     initial begin
46         clk = 0;
47         rst_n = 0;
48         enable_bs = 0;
49         load_in = 0;
50         load_out = 0;
51         din = 8'h00;
52
53         #100;
54         rst_n = 1;
55
56         repeat (16) begin
57             @(negedge clk);
58             load_in = 1;
59             din = $random;
60         end
61
62         @(negedge clk);
63         load_in = 0;
64         enable_bs = 1;
65
66         // 等十個週期經過 cs
67         repeat(10) @(negedge clk);
68         enable_bs = 0;
69
70         load_out = 1;
71         // 依序送出 16 個 byte
72         repeat (15) begin
73             @(negedge clk);
74             load_out = 1;
75         end
76
77         @(negedge clk);
78         load_out = 0;
79
80         #100;
81         $finish;
82     end
83 endmodule

```

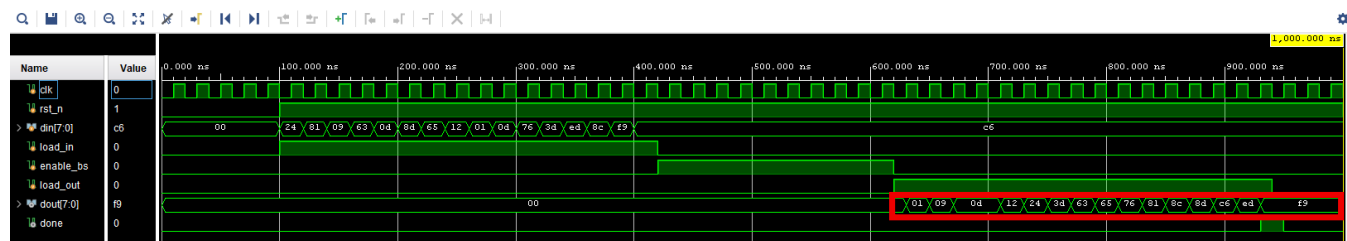
Simulation :

在模擬方面，我先用 \$random 的方式把 din 送入隨機的 16 組 byte 當作準備排序的資料，接著等待 10 個 clock cycle 為 Bitonic Sorter 16 inputs 的運算時間，最後，再依序送出 16 個 byte (由 min ~ max)，完整所以 $16 + 10 + 16 = 42$ 個 clock cycle，但是由於子模組所需的控制訊號還沒有用 controller 來管控，所以在這裡

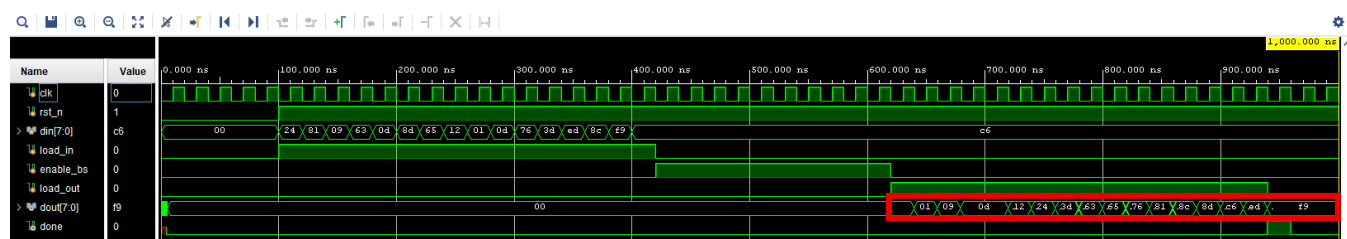
的 testbench 必須自己寫對應的控制訊號開啟與關閉時間。

以下 16 組輸入 byte 分別為：8'h24、8'h81、8'h09、8'h、63、8'h0d、8'h8d、8'h65、8'h12、8'h01、8'h0d、8'h76、8'h3d、8'hed、8'h8c、8'hf9、8'hc6，在完成排序後可得到 dout 會由 min ~ max 依序輸出 8'h01、8'h 09、8'h 0d、8'h 0d、8'h 12、8'h 24、8'h 3d、8'h 63、8'h 65、8'h 76、8'h 81、8'h 8c、8'h 8d、8'h c6、8'h ed、8'h f9。

Behavior Simulation：



Post-Route Simulation：



9. Controller

接著，為了要把 datapath 所需的控制訊號統一管理，必須要寫一個 controller 來控制，這裡因為各子模組 (SIPO_16x8、Bitonic Sorter 16 input、PISO_16x8) 所需的運算周期都是固定的，所以在 controller 上我就沒有用 FSM 的寫法，只需要用簡單的 count 計數器來描述即可，以下為我寫的 verilog 程式碼。

Verilog：

```

1  `timescale 1ns / 1ps
2  //////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 2025/11/09 15:01:59
7  // Design Name:
8  // Module Name: controller
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 //////////////////////////////////////
21
22
23 module controller(
24     input wire clk,
25     input wire rst_n,
26     input wire start,
27     output reg load_in,
28     output reg enable_bs,
29     output reg load_out
30 );
31
32     reg [5:0] count;
33     always @(posedge clk or negedge rst_n)begin
34         if(!rst_n)begin
35             load_in    <= 0;
36             enable_bs  <= 0;
37             load_out   <= 0;
38             count      <= 6'b0;
39         end
40         else if (start | count != 6'b0)begin
41             if (count >= 0 & count < 16)begin
42                 load_in    <= 1'b1;
43                 enable_bs  <= 1'b0;
44                 load_out   <= 1'b0;
45                 count      <= count +1;
46             end
47             else if (count >= 16 & count < 26)begin
48                 load_in    <= 1'b0;
49                 enable_bs  <= 1'b1;
50                 load_out   <= 1'b0;
51                 count      <= count +1;
52             end
53             else if (count >= 26 & count < 42)begin
54                 load_in    <= 1'b0;
55                 enable_bs  <= 1'b0;
56                 load_out   <= 1'b1;
57                 count      <= count +1;
58             end
59             else begin
60                 load_in    <= 1'b0;
61                 enable_bs  <= 1'b0;
62                 load_out   <= 1'b0;
63                 count      <= 6'b0;
64             end
65         end
66     end
67 endmodule

```

Testbench :

```

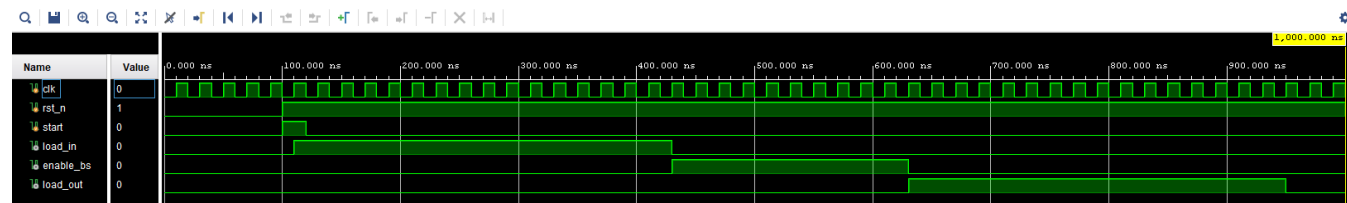
1  `timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 2025/11/09 15:18:22
7  // Design Name:
8  // Module Name: tb_controller
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21
22
23 module tb_controller;
24     reg        clk;
25     reg        rst_n;
26     reg        start;
27     wire        load_in;
28     wire        load_out;
29     wire        enable_bs;
30
31
32     controller dut (
33         .clk(clk),
34         .rst_n(rst_n),
35         .start(start),
36         .load_in(load_in),
37         .load_out(load_out),
38         .enable_bs(enable_bs)
39     );
40
41     always #10 clk = ~clk;
42     initial begin
43         clk = 0;
44         rst_n = 0;
45         start = 0;
46
47         #100;
48         rst_n = 1;
49         start = 1;
50
51         #20;
52         start = 0;
53
54         repeat (42) @(negedge clk); // 等 42 個週期觀察結果 (16+10+16)
55
56         #100
57         $finish;
58     end
59 endmodule

```

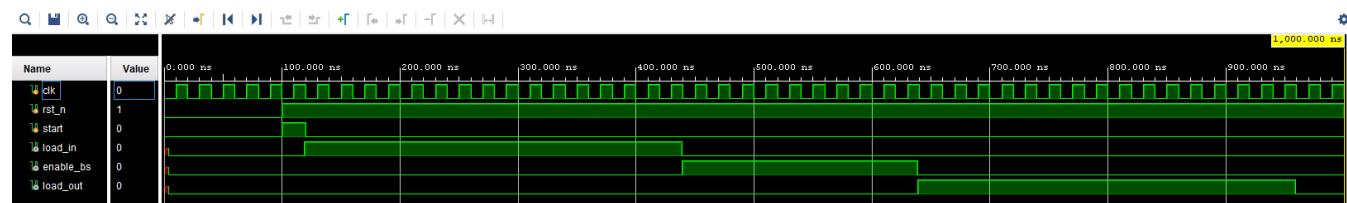
Simulation :

在模擬上，rst_n 結束後，我讓 start 輸入訊號為 1，然後觀察後續 42 個 clock cycle 有沒有在正確的時間內開啟正確的控制訊號，以此作為驗證。

Behavior Simulation :



Post-Route Simulation :



10. Complete Sorter 16 inputs 8bits each

最後，把剛才的 datapath 與 controller 模組結合在一起，就可以完成這次的 project (Sorting Network)，總 I/O pin 腳為 20 個，有確實符合 project 所限制的 32 個以內，以下為我寫的最終 verilog 程式碼。

Verilog :

```

1  `timescale 1ns / 1ps
2  //////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 2025/11/09 15:31:42
7  // Design Name:
8  // Module Name: complete_sorter_16x8
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 //////////////////////////////////////
21
22

```

```

23 module complete_sorter_16x8(
24     input wire clk,
25     input wire rst_n,
26     input wire start,
27     input wire [7:0] din,
28     output wire [7:0] dout,
29     output wire done
30 );
31
32     wire load_in;
33     wire load_out;
34     wire enable_bs;
35     wire clk_b;
36     assign clk_b = ~clk;
37     datapath d0(.clk(clk),
38                 .rst_n(rst_n),
39                 .load_in(load_in),
40                 .load_out(load_out),
41                 .enable_bs(enable_bs),
42                 .din(din),
43                 .dout(dout),
44                 .done(done)
45     );
46     controller c0(.clk(clk_b),
47                  .rst_n(rst_n),
48                  .start(start),
49                  .load_in(load_in),
50                  .load_out(load_out),
51                  .enable_bs(enable_bs)
52     );
53 endmodule

```

Testbench :

```

1  `timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 2025/11/09 15:37:14
7  // Design Name:
8  // Module Name: tb_complete_sorter_16x8
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21
22
23 module tb_complete_sorter_16x8;
24     reg        clk;
25     reg        rst_n;
26     reg        start;
27     reg [7:0]  din;
28     wire [7:0] dout;
29     wire       done;
30
31     complete_sorter_16x8 dut (
32         .clk(clk),
33         .rst_n(rst_n),
34         .start(start),
35         .din(din),
36         .dout(dout),
37         .done(done)
38     );

```



```

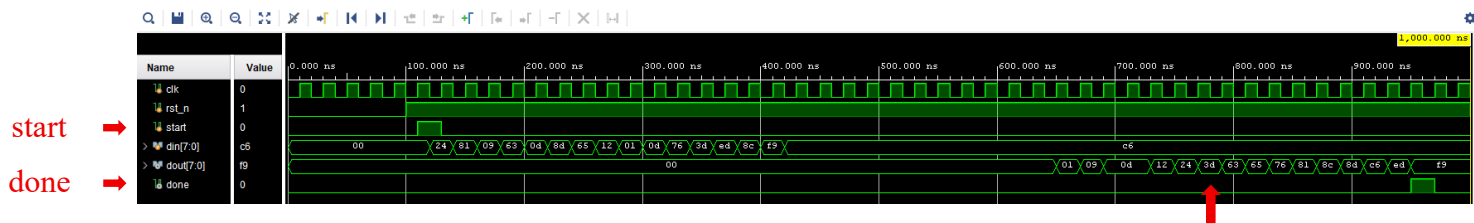
40     always #10 clk = ~clk;
41     initial begin
42         clk = 0;
43         rst_n = 0;
44         start = 0;
45         din = 0;
46
47         #100;
48         rst_n = 1;
49         @(posedge clk);
50         start = 1;
51         @(negedge clk);
52         din = $random;
53         @(posedge clk);
54         start = 0;
55
56         repeat (15) begin
57             @(negedge clk);
58             din = $random;
59         end
60         repeat (26) @(negedge clk); // 等 26 個週期觀察結果 (10+16)
61
62         #100
63         $finish;
64     end
65 endmodule

```

Simulation :

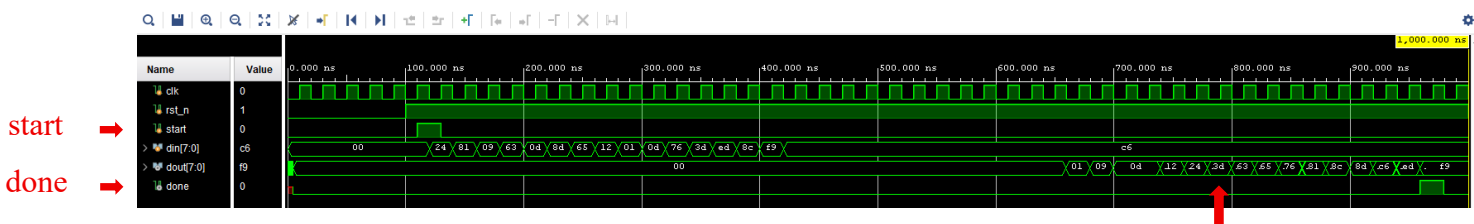
在模擬上，我一樣依照之前的 \$random 寫法，給 din 依序輸入隨機的 16 組 byte，接著由於控制訊號已經由 controller 管理，所以就等待 16 + 10 + 16 = 42 個 clock cycle 過後，直接觀察最終的排序結果，此時可以看到 dout 會依序由小到大傳出：8'h01、8'h09、8'h0d、8'h0d、8'h12、8'h24、8'h3d、8'h63、8'h65、8'h76、8'h81、8'h8c、8'h8d、8'hc6、8'hed、8'hf9。

Behavior Simulation :



dout 已排序(min ~ max)

Post-Route Simulation :



dout 已排序(min ~ max)

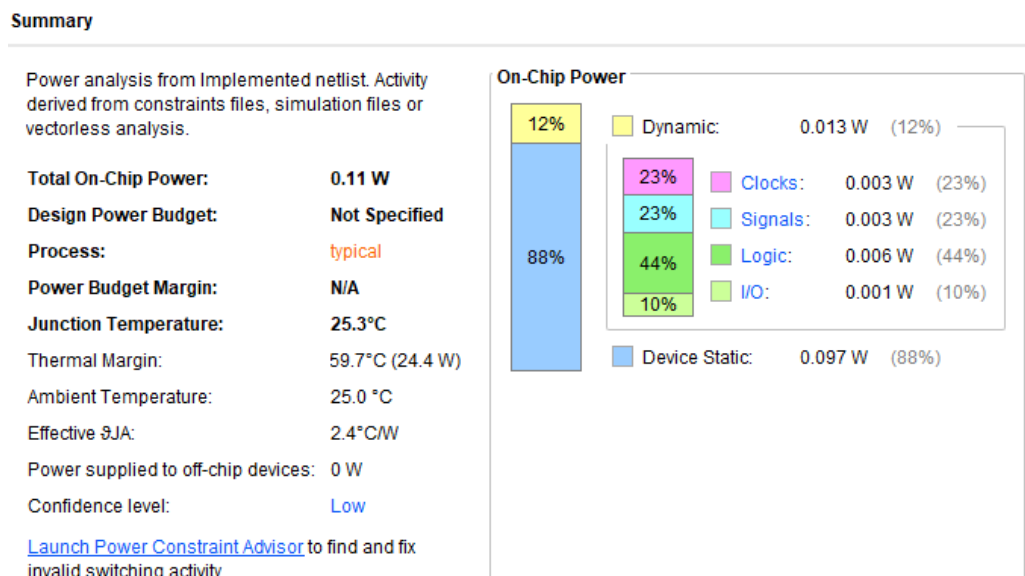
Performance

Name	Waveform	Period (ns)	Frequency (MHz)
clk	{0.000 5.000}	10.000	100.000

Design Timing Summary			
Setup		Hold	Pulse Width
Worst Negative Slack (WNS): 2.233 ns		Worst Hold Slack (WHS): 0.167 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): 0.000 ns		Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0		Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 2851		Total Number of Endpoints: 2851	Total Number of Endpoints: 1431
All user specified timing constraints are met.			

在效能方面，可以達到 100 MHz 的工作頻率。從 timing report 可以看到，Setup 的 Worst Negative Slack 為 2.233 ns、Hold 的 Worst Hold Slack 為 0.167ns，Worst Pulse Width Slack 也有 4.500 ns，都沒有 timing violation，可以穩定運作。再加上前面分析過，完成一次 16×8 bits 的排序 (含輸入輸出) 一共需要 42 個 clock cycle，因此在目前設定下，完成一次完整排序大約只要 $42 \times 10 \text{ ns} \approx 0.42 \mu\text{s}$ 就可以輸出由最小到最大的結果。

Power Consumption



在功耗方面，結果顯示 Total On-Chip Power 約為 0.11 W。其中，Device Static 約 0.097W (約佔 88%)，而 Dynamic Power 約 0.013W (約佔 12%)。在動態功耗的部分，又可分為 Clocks 0.003 W (23%)、Signals 0.003 W (23%)、Logic 0.006 W (44%) 以及 I/O 約 0.001 W (10%)。由於這次專題的資料寬度與硬體規模都不算大，加上控制電路相對單純，因此整體動態功耗相當小。從報告中的 Junction Temperature 25.3°C 以及約 59.7°C 的 Thermal Margin 也可以看出，在目前設定下對實際 FPGA 實作不會造成明顯的散熱壓力。

Hardware Cost

Tcl Console																					Messages		Log		Reports		Design Runs		x		DRC		Methodology		Power		Timing				?		_		□		□	
Q		≡		⏏		⏮		⏪		⏩		⏭		+		%																																
Name		Constraints		Status		^1		WNS		TNS		WHS		THS		WBSS		TPWS		Total Power		Failed Routes		Methodology		QoA Score		QoR Suggestions		LUT		FF		BRAM		URAM		DSP		Start		Elapsed		Run Strategy				
✓ synth_1		constrs_1		synth_design Complete!																										1006		1430		0		0		0		11/9/25, 4:04 PM		00:00:22		Vivado Synthesis De				
✓ impl_1		constrs_1		route_design Complete!				2.233		0.000		0.167		0.000				0.000		0.110		0		19 Warn						1005		1430		0		0		0		11/9/25, 4:04 PM		00:00:42		Vivado Implementati				

在硬體資源方面，完成合成與繞線後大約使用約 1005 個 LUT 與 1430 個 FF。以本次實驗所使用的 FPGA 規模來看，這樣的使用量只佔整體邏輯資源一點點，資源上仍相當充裕，且此設計在硬體角度上是合理的。

Discussion

這次高等演算法的 Final Project 讓我更深入地理解硬體電路在平行化資料處理與結構化設計上的應用。相較於上學期實作的 multi-cycle CPU，這次我選擇設計一個 Bitonic Sorter (16 inputs, 8 bits each)，這讓我知到如何以硬體的設計方式去實現高效能的排序網路。在過程中，我需要從硬體的角度理解 Bitonic Sort 的層級結構，同時了解平行處理在排序網路裡每個 stage 的資料流動與時序。

在設計階段，我將整體架構拆分為多個模組：包含 SIPO_16x8、Bitonic Sorter 16 inputs、以及 PISO_16x8。SIPO_16x8 與 PISO_16x8 負責資料的輸入與輸出，而核心的排序模組 Bitonic Sorter 16 inputs 則由多層的 Bitonic Merger 所組成。這樣的模組化設計讓我對整個系統的結構更加了解，也方便之後在 simulation 時的驗證。過程中最具挑戰的部分，我認為是在設計階段，確認好各模組所需的控制訊號與輸入輸出是什麼，並保持各模組之間正確資料的傳遞與控制，我覺得是這次 project 的關鍵之處。另外，在實際寫 verilog 時，我深刻體會模組化的重要性。Bitonic Sorter 雖然邏輯上規則，但層數多且連線複雜，若沒有從一開始的 Bitonic Merger 小模組就清楚設計好，程式就會變得難以維護與了解，也因此必須理解每個模組之間的關係。透過這次的 project，我也因此提升了自己在程式撰寫的能力。

最後，這次的 Sorting Network 讓我了解到平行運算在硬體上的優勢。相比軟體層的序向處理，Sorting Network 可以在固定的周期內完成大量的 Compare and Swap 操作，展現出平行化的高速特性。總之，這次的 project，我認為不只是一次 IP design 的練習，更是一次讓我從演算法思維轉換到硬體架構思維的經驗，我覺得對我未來有重要的幫助。